

Real Time Accident Alert System In Hill Region Using Lora Technology

PROJECT REPORT PHASE-II (15PRBEC61)

Submitted by

ESHWARI.M (22603655)
MOHANA SUNDARAM.S (22603656)

*In partial fulfilment for the award of the degree
of*

BACHELOR OF ENGINEERING
In
ELECTRONICS AND COMMUNICATION ENGINEERING

Under the Guidance of

Dr. ARULSTEPHEN.C
Assistant Professor,
Department of Electronics and Communication Engineering,
School of Engineering, Vels Institute of Science, Technology &
Advanced Studies (VISTAS), Chennai – 600117



Department of Electronics and Communication Engineering,
School of Engineering, Vels Institute of Science, Technology &
Advanced Studies (VISTAS), Chennai – 600117

NOVEMBER 2025

BONAFIDE CERTIFICATE

Certified that this project report titled “**Real Time Accident Alert System in Hilly Region using LoRa Technology**” is the Bonafide work Eshwari M(Reg.no.22603655) and Mohana Sundaram S (Reg.no.22603656) who carried out the project work under my supervision. Certified further that to the best of my knowledge the work reported here in does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

**Head of Department
Dr. A. Vijayalakshmi
Professor
Department of ECE
VISTAS, Chennai-600117**

SIGNATURE

**Project Supervisor
Dr. ArulStephen.C
Assistant Professor
Department of ECE
VISTAS, Chennai-600117**

**For the project work and Viva-Voce Examination held on.....VELS
Institute of Science Technology&Advanced Studies (VISTAS), Chennai-600117.**

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We wish to express our heartfelt gratitude and thanks to our founder chairman Shri.**Ishari K. Ganesh** for providing a good infrastructural support with all kinds of lab facilities to do the project work successfully.

We wish to express my heartfelt thanks to **Dr. A. JothiMurugan**, pro-chancellor (P&D), **Dr. Preethaa Ganesh**, Vice President, VELS Group of Institutions, **Dr. S. Sriman Narayanan**, Vice Chancellor, **Dr.P.Saravanan**, Registrar and **Dr. A. Udhayakumar** Controller of Examinations for providing us necessary facilities to accomplish this project work and permitting us to undertake the research programme.

We wish to express our sincere thanks to **Dr. C.Arun**, Dean, School of Engineering, Vels Institute of Science Technology & Advanced Studies (VISTAS) for his good counsel, encouragement, valuable suggestions and support rendered at all times of our project work.

We wish to express our sincere thanks to **Dr. A. Vijayalakshmi**. Head of the Department, Department of Electronics and Communication Engineering, VELS Institute of Science Technology & Advanced Studies (VISTAS), for her valuable guidance and the facilities provided to us.

We are indebted to our project Coordinators **Dr. V. Rajendran**, Professor/Director and **Dr.Arul stephen**, Associate Professor, Department of ECE for their motivation, guidance and valuable suggestions given to us throughout our discussions and reviews. This gave us the driving force for us to proceed with our project work with new innovative methods, we are grateful to them to move forward in our career and life.

We also wish to express our sincere thanks to our Supervisor **Dr. Arul Stephen C**, Assistant Professor, Department of Electronics and Communication Engineering, VISTAS for providing valuable suggestions to us to carry out this project work by her constant monitoring, continuous encouragement at various stages of the work throughout our discussions and reviews.

We would also like to thank all teaching and non-teaching staff of our department for their unfailing support for our project work to this extent.

Abstract

Accident detection and timely emergency response are particularly challenging in hilly regions due to rugged terrain, unreliable cellular networks, and the difficulty in pinpointing exact accident locations. These factors often lead to critical delays in emergency services, increasing the severity of accidents. To overcome these challenges, this study proposes a real-time accident alert system that utilizes LoRa (Long Range) communication technology, integrated with various sensors, GPS, GSM, and the Blynk mobile application. The system features an MPU6050 tilt sensor to detect vehicle inclination, an SW-420 vibration sensor to sense abrupt shocks, and a NEO-6M GPS module to provide accurate geolocation data. These components are connected to an Arduino Uno microcontroller, forming the transmitter unit installed in the vehicle. On the receiver side, a NodeMCU ESP8266 module processes the incoming data via LoRa. In the event of an accident, the system automatically sends SMS alerts to predefined emergency contacts, nearby hospitals, and police stations via a GSM module. The alerts include real-time GPS coordinates and a clickable Google Maps link for easy location tracking. Additionally, the Thinksboard app serves as a user-friendly interface, displaying live sensor data, vehicle status, and accident notifications. The proposed system offers a low-power, long-range communication solution suitable for remote and mountainous regions. It significantly enhances the speed and reliability of emergency response, providing a scalable and practical approach to improving road safety in challenging terrains.

Keywords:

Accident detection, LoRa, Arduino Uno, NodeMCU ESP8266, GPS, GSM, Blynk app, hilly regions, real-time alert system, emergency response, vehicle safety, SW-420, MPU6050.

INDEX

CHAPTER NO	TITLE	PAGE NO
1	INTRODUCTION TO REAL TIME ACCIDENT ALERT SYSTEM IN HILL REGION USING LORA TECHNOLOGY	11
1.1	General Overview	11
2	PROJECT SYNOPSIS	13
2.1	Aim	13
2.2	Objectives	13
2.3	Scope of the project	15
3	LITERATURE REVIEW	17
4	HARDWARE COMPONENTS	19
4.1	Arduino R3	19
4.1.1	Overview	19
4.1.2	Specifications	19
4.1.3	Functionality	19
4.1.4	Arduino R3 Applications	23
4.2	SW-420 Vibration Sensor	24
4.2.1	Overview	24
4.2.2	Specifications	24
4.2.3	Functionality	25
4.3	MPU6050 Gyro-Accelerometer	26
4.3.1	Overview	26
4.3.2	Specifications	27
4.3.3	Functionality	27
4.4	NEO-6M GPS Module	30
4.4.1	Overview	30

4.4.2	Specifications	30
4.4.3	Functionality	31
4.5	LoRa Module	34
4.5.1	Overview	34
4.5.2	Specifications	35
4.5.3	Functionality	35
4.6	NodeMCU ESP8266	37
4.6.1	Overview	37
4.6.2	Specifications	38
4.6.3	Functionality	38
4.7	GSM900A Module	40
4.7.1	Overview	40
4.7.2	Specifications	41
4.7.3	Functionality	42
4.8	Assembly Instructions	44
4.8.1	Transmitter Node Assembly	44
4.8.2	Receiver Node Assembly	47
5	METHODOLOGY	49
5.1	System Design	49
5.2	Sensor Integration	50
5.3	Communication Setup	50
5.4	Testing and Calibration	51
5.5	Implementation of Alert Mechanism	51
5.6	Data Collection and Analysis	52
6	SOFTWARE DESCRIPTION	53
6.1	Overview of Software Architecture	53
6.2	Code Implementation	53
6.3	Arduino Code (Transmitter Node)	54
6.4	Things Board Configuration	54
6.5	User Interface Design	55
7	RESULT AND OUTPUT	56

7.1	Testing Scenarios	57
7.2	Performance Metrics	57
7.3	Data Analysis	57
7.4	Thingsboard Cloud	58
7.4.1	Thingsboard Cloud (Before accident)	58
7.4.2	Thingsboard Cloud (After accident)	59
7.5	Arduino IDE Serial Monitor	60
7.5.1	Arduino IDE Serial Monitor (Transmitter node)	61
7.5.2	Arduino IDE Serial Monitor (Receiver node)	63
7.6	SMS alert on phone	65
8	CONCLUSION	67
8.1	Summary of Findings	67
8.2	Implications for Road Safety	68
8.3	Recommendations for Future Work	68
9	<i>REFERENCES</i>	69
10	APPENDIX	71
10.1	Transmitter code	71
10.2	Receiving code	75

LIST OF FIGURES

Figure No.	Title of Figure	Page No.
Figure 2.1	LoRa-Based Accident Alert System Overview	16
Figure 4.1	Arduino Pin Configuration	23
Figure 4.2	SW-420 Vibration Sensor Pin Configuration	24
Figure 4.3	SW-420 Vibration Sensor Interface Diagram	25
Figure 4.4	MPU6050 Gyro-Accelerometer Pin Configuration	26
Figure 4.5	MPU6050 Gyro-Accelerometer Interface Diagram	29
Figure 4.6	NEO-6M GPS Module Pin Configuration	30
Figure 4.7	NEO-6M GPS Module Interface Diagram	33
Figure 4.8	LoRa Module Pin Configuration	34
Figure 4.9	LoRa Module Interface Diagram	36
Figure 4.10	NodeMCU ESP8266Pin Configuration	37
Figure 4.11	GSM900A Module Pin Configuration	40
Figure 4.12	GSM800L Module Interfacing Diagram	43
Figure 7.1	Thingsboard Cloud (Before Accident)	59
Figure 7.2	Thingsboard Cloud (After Accident)	60
Figure 7.3	Transmitter Node	62
Figure 7.4	Arduino IDE Serial Monitor (Transmitter Node)	63
Figure 7.5	Receiver Node	63
Figure 7.6	Arduino IDE Serial Monitor (Receiver Node)	64
Figure 7.6	SMS & Call alert after accident (Receiver Node)	64
Figure 7.6	SMS Alert	65
Figure 7.8	Accident Location Coordinates from GPS	66

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
Table 7.1	Performance Metrics	57

LIST OF ABBREVIATIONS

LORA	Long Range (Radio)
GPS	Global Positioning System
GSM	Global System for Mobile Communication
IOT	Internet Of Thing
PWM	Pulse Width Modulation
I2C	Inter -Integrated Circuit
SPI	Serial Peripheral Interface
UART	Universal Asynchronous Receiver-Transmitter
ESP8266	A Wi-Fi microchip with full TCP/IP stack
MPU6050	Motion Processing Unit (Gyroscope + Accelerometer)
WSN	Wireless Sensor Network
MITM	Man-in-the-Middle (attack)
SAR	Search and rescue
NB-IOT	Narrowband Internet of Things
LED	Light Emitting Diode (implied, though not used directly)

CHAPTER 1

INTRODUCTION

1.1 GENERAL OVERVIEW

The increasing incidence of road accidents, particularly in hilly regions, presents significant challenges to public safety and emergency response systems. The unique geographical features of these areas often lead to more severe consequences in the event of accidents, making timely detection and response critical. In India, for instance, road accidents claimed the lives of over 1.68 lakh individuals in 2022, underscoring the urgent need for effective solutions to enhance road safety.

To address these pressing issues, a real-time accident alert system has been developed, leveraging advanced technologies such as Long Range (LoRa) communication, Global Positioning System (GPS), and various sensors. This innovative system is designed to continuously monitor vehicle conditions, utilizing tilt sensors (MPU6050) to detect potential rollovers, vibration sensors (SW-420) to identify abrupt impacts, and GPS technology (NEO-6M) to provide accurate location data. By integrating these components, the system can effectively assess the vehicle's status and detect accidents in real-time.

In the event of an accident, the system automatically triggers alerts, transmitting vital information to nearby helping stations and first responders through LoRa technology. This communication method is particularly advantageous in hilly terrains, as it ensures efficient data transmission over long distances while consuming low power, making it ideal for areas where cellular networks may be unreliable or unavailable. Additionally, the system incorporates a GSM module to send SMS alerts to predefined contacts, including hospitals and police stations, further enhancing emergency response capabilities.

The user interface is facilitated through the Blynk mobile app, which provides a user-friendly platform for vehicle operators and emergency responders to access real-time tracking and notifications. The app displays critical information such as vehicle location, tilt angle, and vibration levels, allowing for quick assessment and action in emergency situations.

By leveraging these advanced technologies, the proposed accident alert system aims to significantly enhance road safety in hilly regions. The integration of real-time accident detection, precise location tracking, and efficient communication channels is expected to reduce response times and improve the overall effectiveness of emergency services. Ultimately, this innovative approach represents a significant advancement in addressing the challenges associated with road safety in difficult terrains, contributing to the reduction of accidents and their associated consequences. Through ongoing developments and enhancements, this system has the potential to play a crucial role in the future of intelligent transportation systems and emergency response mechanisms.

CHAPTER 2

PROJECT SYNOPSIS

2.1 AIM

To design and implement a Real-Time Accident Alert System using LoRa Technology for hilly regions, capable of detecting vehicle accidents through tilt, vibration, and temperature sensors, and transmitting GPS-based location data to emergency responders via LoRa and GSM networks, ensuring timely assistance in remote or low-network areas.

2.2 OBJECTIVES

The primary objectives of the Real-Time Accident Alert System in Hilly Regions Using LoRa Technology are as follows:

1. Enhance Accident Detection:

- Develop a system that utilizes advanced sensors (tilt, vibration, and GPS) to detect accidents in real-time, ensuring prompt identification of incidents.

2. Implement Long-Range Communication:

- Integrate LoRa (Long Range) technology to facilitate efficient and reliable communication of accident alerts over long distances, particularly in areas with limited cellular network coverage.

3. Provide Accurate Location Data:

- Utilize GPS technology to provide precise location coordinates of the accident site, enabling emergency responders to reach the scene quickly and efficiently.

4. Improve Emergency Response Times:

- Design the system to automatically send alerts to nearby emergency services, hospitals, and first responders, thereby reducing the time taken to respond to accidents.

5. Create a User-Friendly Interface:

- Develop a mobile application (using Blynk) that allows vehicle operators and emergency responders to access real-time data, including vehicle status, location, and alerts, enhancing situational awareness.

6. Facilitate Data Monitoring and Analysis:

- Implement features that allow for continuous monitoring of vehicle conditions and the collection of data for analysis, which can be used to improve the system and inform future safety measures.

7. Promote Road Safety Awareness:

- Raise awareness about the importance of road safety in hilly regions through the deployment of the system, potentially leading to behavioral changes among drivers and increased vigilance.

8. Support Future Research and Development:

- Establish a foundation for further research into intelligent transportation systems and emergency response mechanisms, including the potential integration of machine learning algorithms for enhanced accident detection accuracy.

9. Evaluate System Performance:

- Conduct thorough testing and evaluation of the system's performance in various scenarios to ensure reliability, accuracy, and effectiveness in real-world applications.

10. Contribute to Policy Development:

- Provide insights and data that can inform policymakers and stakeholders in developing regulations and initiatives aimed at improving road safety in hilly regions.

2.3 SCOPE OF THE PROJECT

The scope of the Real-Time Accident Alert System in Hilly Regions Using LoRa Technology focuses on enhancing road safety in challenging terrains, particularly in high-risk areas like Himachal Pradesh and Jammu & Kashmir.

The project integrates advanced technologies, including LoRa communication for long-range data transmission, GPS for accurate location tracking, and various sensors to detect accidents in real-time. It involves the development of a cohesive system comprising hardware components, a user-friendly mobile application for monitoring and alerts, and comprehensive testing to evaluate performance and reliability.

Additionally, the project aims to promote road safety awareness among drivers and local communities while laying the groundwork for future research in intelligent transportation systems. Limitations include a focus on technical aspects without addressing broader policy changes or infrastructure improvements. The project is designed to be completed within a specified timeframe, ensuring timely progress and effective outcomes.

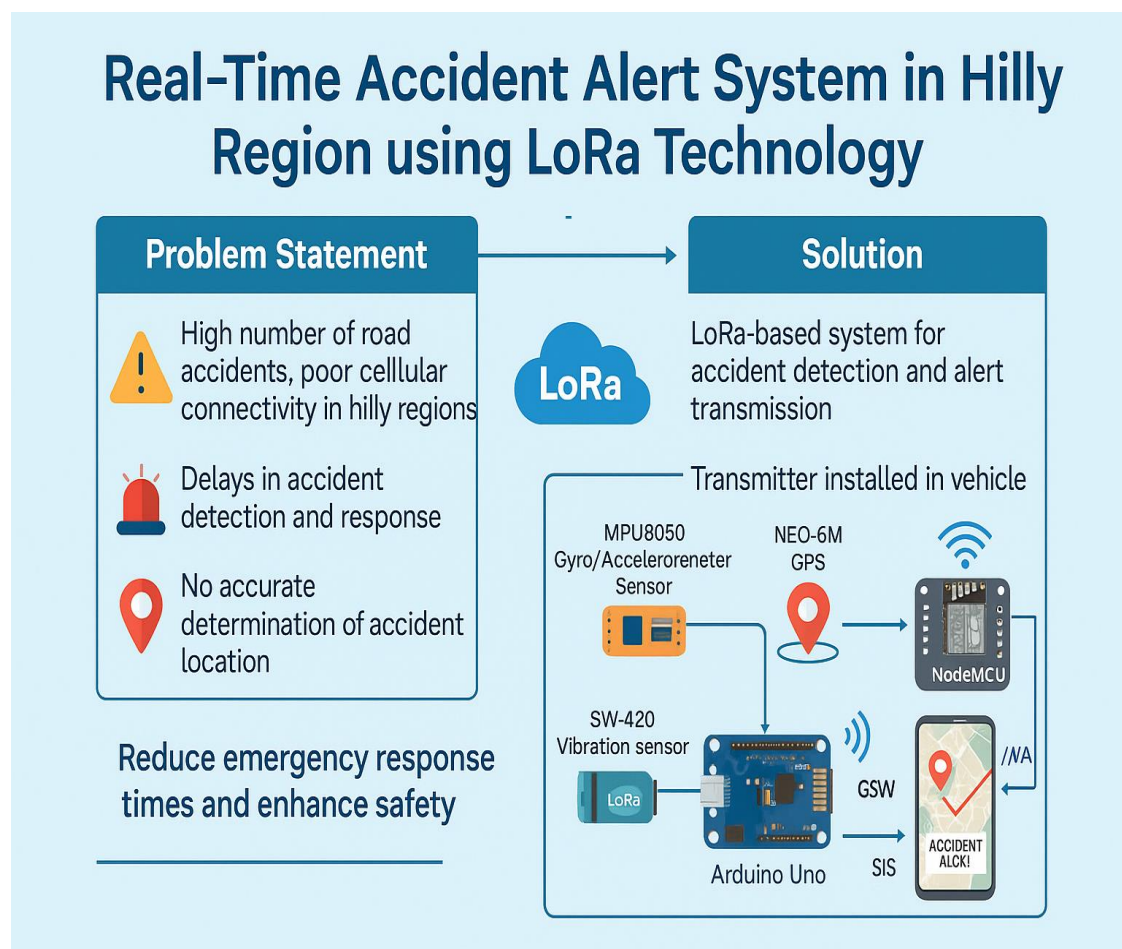


Figure 2.1

LoRa-Based Accident Alert System Overview

CHAPTER 3

LITERATURE REVIEW

1. The literature on road safety technologies highlights a range of innovative solutions aimed at reducing accidents and enhancing emergency response systems. **Overview of Road Safety Technologies** reveals those advancements such as Advanced Driver Assistance Systems (ADAS), Vehicle-to-Everything (V2X) communication, and Intelligent Transportation Systems (ITS) have significantly improved vehicle safety and traffic management. ADAS features, including collision avoidance and lane departure warnings, assist drivers in making safer decisions, while V2X communication facilitates real-time information exchange between vehicles and infrastructure, enhancing situational awareness. Additionally, ITS employs traffic management and incident detection systems to optimize traffic flow and respond swiftly to accidents.

2. **Previous Research on Accident Detection Systems** indicates that various methodologies have been explored to improve accident detection accuracy and response times. Studies have demonstrated the effectiveness of sensor-based systems, which utilize accelerometers, gyroscopes, and GPS technology to monitor vehicle dynamics and detect accidents in real-time. Research has also highlighted the integration of machine learning algorithms to analyse data from these sensors, enabling more precise accident predictions and reducing false positives.

3. The **Role of IoT in Road Safety** has emerged as a transformative force in enhancing road safety measures. The Internet of Things (IoT) enables the

interconnection of vehicles, infrastructure, and emergency services, allowing for seamless communication and data sharing. IoT applications in road safety include real-time monitoring of vehicle conditions, automated alerts to emergency responders, and data analytics for identifying high-risk areas. This interconnectedness fosters a proactive approach to accident prevention and response.

4. Despite these advancements, there are notable **Limitations of Existing Systems** that hinder their effectiveness. Many traditional accident detection systems rely on manual reporting or basic sensor technologies, leading to delays in detection and response. Additionally, the lack of reliable communication infrastructure in remote or hilly regions poses significant challenges for timely notifications. Existing systems may also struggle with issues such as false alarms, limited coverage, and inadequate integration with emergency services, highlighting the need for more robust and comprehensive solutions to enhance road safety.

CHAPTER 4

HARDWARE COMPONENTS

The Real-Time Accident Alert System in Hilly Regions utilizes several key components, each playing a crucial role in ensuring effective accident detection and communication. Below is a detailed description of each component, including the updated specifications for the Arduino R3 .

4.1 ARDUINO R3

4.1.1 Overview: The Arduino R3 serves as the main controller of the system, acting as the central processing unit that integrates and manages data from various sensors.

4.1.2 Specifications:

- Operating Voltage: 5V
- Input Voltage: 7-12V
- Digital I/O Pins: 14 (6 provide PWM output)
- USB Interface: ATmega16U2
- Memory: 2 KB SRAM, 32 KB Flash memory

4.1.3 Functionality: The Arduino platform is a versatile and widely used open-source electronics prototyping environment that allows users to create interactive projects. The Arduino Uno, in particular, is one of the most popular microcontroller boards used in various applications, including the Real-Time Accident Alert System. Below is a detailed overview of the functions and capabilities of the Arduino Uno in this context.

1. Microcontroller Functionality

ATmega328P Microcontroller: The Arduino Uno is based on the ATmega328P

microcontroller, which is responsible for executing the programmed instructions. It features:

- 32 KB of Flash memory for storing code.
- KB of SRAM for variable storage.
- 1 KB of EEPROM for non-volatile data storage.

2. Input/Output (I/O) Operations

Digital I/O Pins: The Arduino Uno has 14 digital pins that can be configured as either input or output. These pins are used to read signals from sensors (input) or send signals to actuators (output).

Input: Read data from sensors like the SW-420 Vibration Sensor or MPU6050 Gyro-Accelerometer.

Output: Control devices such as LEDs or relays based on sensor data.

Analog Input Pins: The board includes 6 analog input pins (A0 to A5) that can read varying voltage levels (0-5V) and convert them into digital values (0-1023). This is particularly useful for reading sensor outputs that provide continuous data, such as temperature or acceleration.

3. Communication Protocols

Serial Communication: The Arduino Uno supports serial communication through its TX (transmit) and RX (receive) pins. This allows for communication with other devices, such as:

GSM Module (GSM800L): Sending SMS alerts or making calls.

LoRa Module: Transmitting accident alerts and location data.

NodeMCU ESP8266: Communicating with the Blynk app for real-time monitoring.

I2C Communication: The Arduino can communicate with multiple devices using the I2C protocol, which is useful for connecting sensors like the MPU6050. This allows for easy integration of multiple components on the same bus.

SPI Communication: The Arduino Uno can also communicate with devices using the SPI (Serial Peripheral Interface) protocol, which is faster than I2C and is often used for high-speed data transfer.

4.Programming Environment

Arduino IDE: The Arduino Integrated Development Environment (IDE) is user-friendly and allows for easy programming of the Arduino Uno. Users can write code in C/C++ and upload it to the board via USB.

Libraries: The Arduino IDE supports a wide range of libraries that simplify the use of various sensors and modules. For example, libraries for the MPU6050 and make it easier to interface with these components.

5. Power Management

Power Supply Options: The Arduino Uno can be powered through various sources, including:

USB connection (5V).

External power supply (7-12V) through the barrel jack.

Vin pin for direct voltage input.

Power Consumption: The board is designed to operate efficiently, making it suitable for battery-powered applications. It can enter a low-power sleep mode to conserve energy when not actively processing data.

6. Real-Time Processing

Loop Function: The Arduino operates on a loop structure, where the `loop()` function continuously executes the code within it. This is essential for real-time applications, such as monitoring sensor data and responding to events (e.g., detecting an accident).

Interrupts: The Arduino Uno can handle interrupts, allowing it to respond to external events (like a button press or sensor trigger) immediately, without waiting for the current loop to finish. This is crucial for timely accident detection.

7.Data Handling and Processing

Data Acquisition: The Arduino Uno can read data from various sensors, process it, and make decisions based on predefined conditions. For example, it can determine if an accident has occurred based on the readings from the vibration sensor and gyroscope.

Data Transmission: After processing the data, the Arduino can send alerts or location information to the LoRa module for communication with emergency services or the Things Board APP.

8. Debugging and Monitoring

Serial Monitor: The Arduino IDE includes a Serial Monitor feature that allows users to view real-time data output from the Arduino

4.1.4 Arduino R3 Applications

Here are some simple points highlighting the applications of the Arduino R3:

IoT (Internet of Things): Connects devices to the internet for data collection and monitoring.

Wearable Technology: Used in fitness trackers to measure health metrics.

Data Logging: Collects and stores data from sensors over time.

Prototyping: Helps create and test prototypes of new products quickly.

Art and Interactive Installations: Creates engaging art pieces that respond to user interaction.

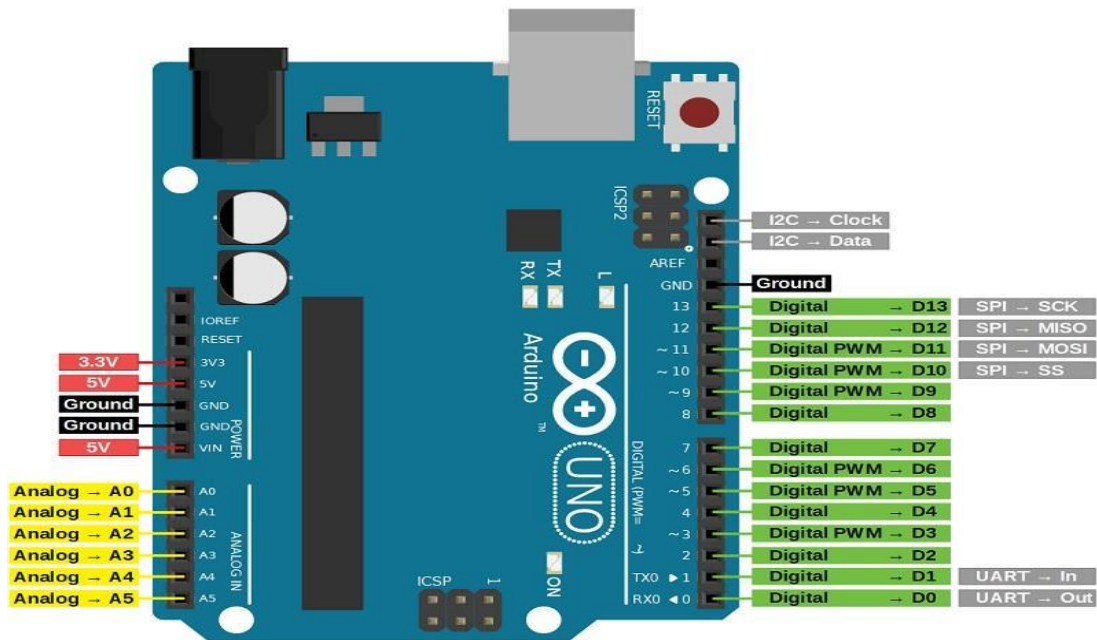


Figure 4.1

Arduino Pin Configuration

4.2 SW-420 VIBRATION SENSOR

4.2.1 Overview: The SW-420 Vibration Sensor is designed to detect vibrations and sudden jolts, which are indicative of potential accidents.

ADITY Vibration Sensor SW420 Module

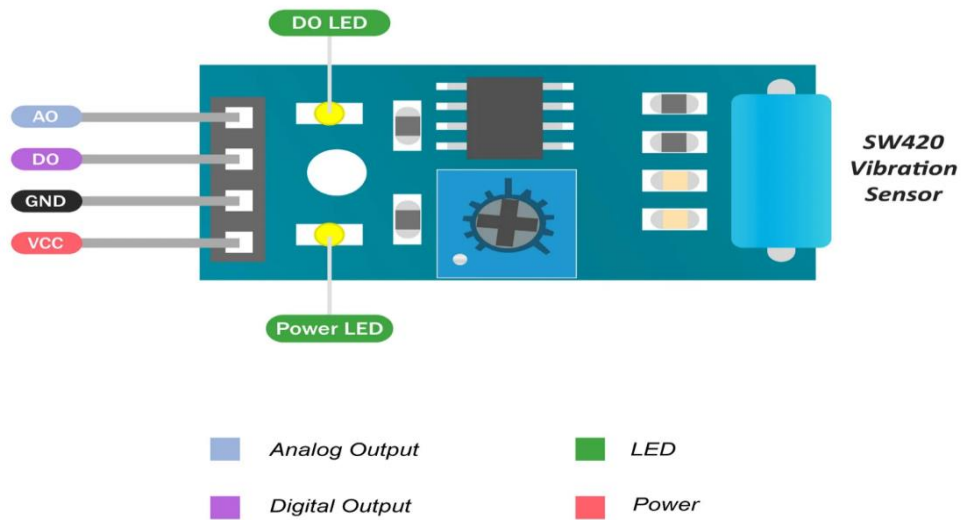


Figure 4.2

SW-420 Vibration Sensor Pin Configuration

4.2.2 Specifications:

- Operating Voltage: 3.3V - 5V
- Current Consumption: <1mA
- Sensitivity Adjustment: Potentiometer for sensitivity adjustment
- Detection Range: Adjustable based on sensitivity settings

4.2.3 Functionality: Detection Mechanism: The sensor operates based on a mechanical switch that closes when vibrations are detected. It consists of a mass that moves in response to vibrations, triggering the switch.

Output Signal: When the sensor detects vibrations above a certain threshold, it

outputs a digital signal (HIGH) to indicate the presence of vibration. If no vibrations are detected, the output remains LOW.

Sensitivity Adjustment: The sensor typically includes a potentiometer that allows users to adjust its sensitivity, enabling it to detect varying levels of vibrations based on specific application requirements.

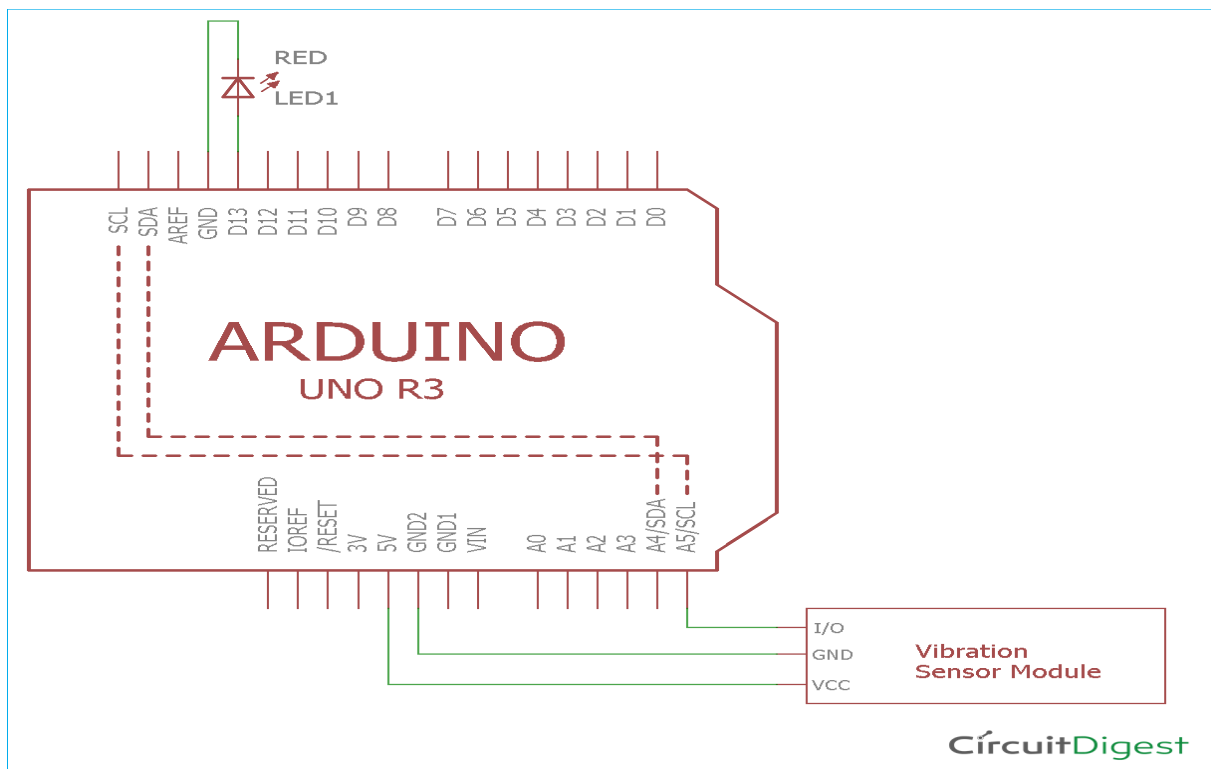


Figure 4.3
SW-420 Vibration Sensor Interface Diagram

4.3 MPU6050 GYRO-ACCELEROMETER

4.3.1 Overview: The **MPU6050** is a compact **6-axis motion sensor** that integrates a **3-axis accelerometer** and a **3-axis gyroscope** into a single chip. It provides accurate information about the **vehicle's orientation, tilt, and movement**, helping to detect

sudden impacts or abnormal motion during accidents. The sensor communicates with the microcontroller through the **I²C interface**, offering **real-time, stable, and low-power operation**, making it ideal for **accident detection and motion analysis** applications.

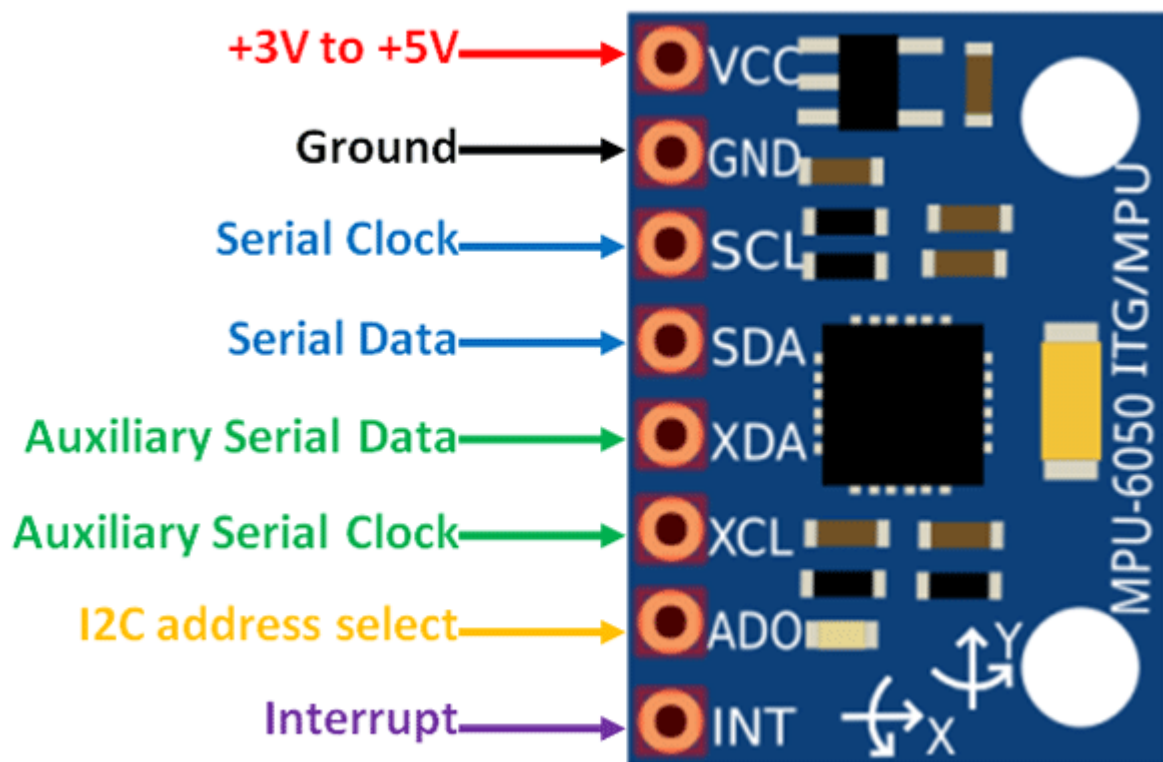


Figure 4.4

MPU6050 Gyro-Accelerometer Pin Configuration

4.3.2 Specifications:

- **Operating Voltage:** 3.3V - 5V
- **Gyroscope Range:** ± 250 , ± 500 , ± 1000 , or ± 2000 degrees per second
- **Accelerometer Range:** $\pm 2g$, $\pm 4g$, $\pm 8g$, or $\pm 16g$

- **Digital Output:** I2C communication protocol.

4.3.2 Functionality:

1. Gyroscope Functionality

Measurement of Angular Velocity: The gyroscope component measures the rate of rotation around the X, Y, and Z axes. This data is crucial for understanding how the object is oriented in space.

Output: The gyroscope provides angular velocity readings in degrees per second ($^{\circ}/s$), allowing for the detection of changes in orientation, such as tilting or rolling.

2. Accelerometer Functionality

Measurement of Linear Acceleration: The accelerometer measures acceleration forces acting on the device along the X, Y, and Z axes. This includes both static forces (like gravity) and dynamic forces (like movement).

Output: The accelerometer provides readings in g (gravitational units), allowing for the detection of changes in speed and direction.

3. Sensor Fusion

Combining Data: The MPU6050 uses sensor fusion algorithms to combine data from both the gyroscope and accelerometer. This process helps to provide a more accurate representation of the object's motion and orientation.

Complementary Filter: The device often employs a complementary filter to balance

the short-term accuracy of the gyroscope with the long-term stability of the accelerometer, resulting in smoother and more reliable output.

4. Temperature Measurement

Onboard Temperature Sensor: The MPU6050 includes an internal temperature sensor that provides temperature readings. This can be useful for applications where temperature variations may affect sensor performance or indicate potential hazards (e.g., fire detection).

5. Digital Output

I2C Communication: The MPU6050 communicates with microcontrollers (like Arduino) via the I2C protocol, allowing for easy integration into various systems. It provides digital output for both gyroscope and accelerometer data, which can be read and processed by the connected microcontroller.

6. Applications

Vehicle Accident Detection and Airbag System

Accelerometers are most widely used in **vehicle safety systems** for accident detection and airbag deployment. They continuously measure the acceleration and deceleration of the vehicle in all directions. When a collision or sudden impact occurs, the sensor instantly detects the rapid change in acceleration and triggers the **airbag system** within milliseconds to protect the passengers from severe injuries. This same principle is also used in advanced **accident alert systems**, where the detected impact data is sent to emergency services for immediate response. As a result, accelerometers play a **life-saving, safety-critical role** in modern vehicles and transportation systems.

Aerospace and Navigation Systems

In the aerospace and navigation field, accelerometers are essential for **measuring motion, tilt, and vibration** of aircraft, spacecraft, and drones. They provide accurate data that helps determine the **orientation, velocity, and stability** of the vehicle in three-dimensional space. Together with gyroscopes and magnetometers, accelerometers form the **Inertial Measurement Unit (IMU)** used for navigation and flight control. This allows aircraft and rockets to maintain stable flight paths even when GPS signals are unavailable. Because they ensure mission accuracy and control, accelerometers are considered **mission-critical components** in aerospace engineering and navigation systems.

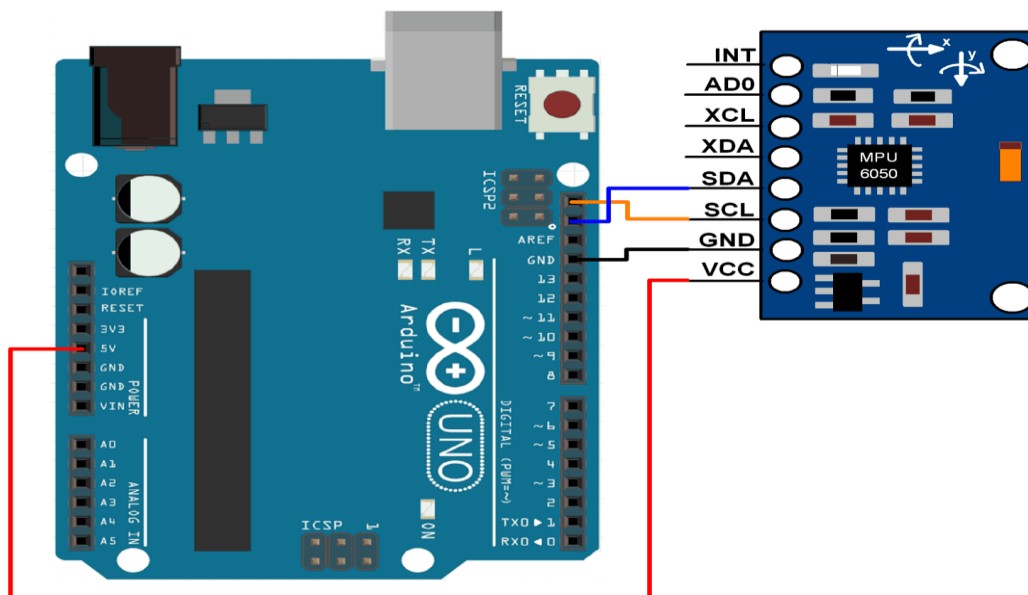


Figure 4.5
MPU6050 Gyro-Accelerometer Interface Diagram

4.4 NEO-6M GPS MODULE

4.4.1 Overview: The NEO-6M GPS Module is responsible for providing accurate location data, essential for notifying emergency responders of the accident site.

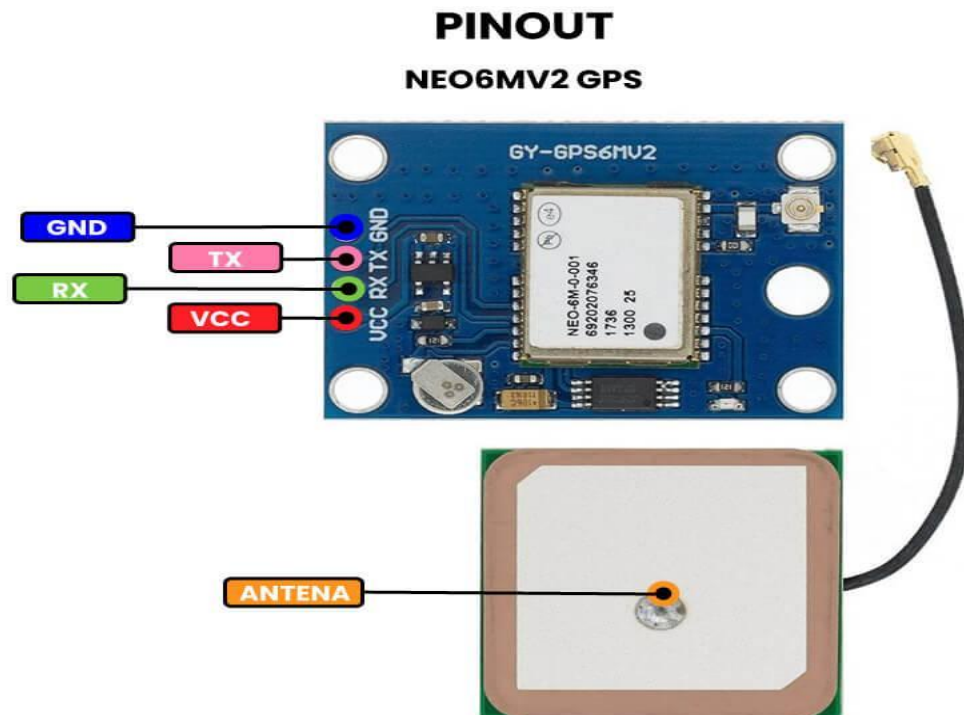


Figure 4.6

NEO-6M GPS Module Pin Configuration

4.4.2 Specifications:

- **Receiver Type:** 50-channel u-blox 6 positioning engines
- **Operating Voltage:** 3.3V - 5V
- **Current Consumption:** Approximately 20mA
 - **Positioning Accuracy:** Approximately 2.5 meters (C/A code, without aiding)
 - **Update Rate:** Up to 5Hz (configurable)

4.4.3 Functionality:

1. Satellite Signal Reception

Multi-Channel Receiver: The NEO-6M can track up to 50 satellites simultaneously, allowing for improved accuracy and reliability in positioning.

GPS Signal Processing: It receives signals transmitted by GPS satellites, which contain information about the satellite's position and the time the signal was sent.

2. Position Calculation

Trilateration: The module uses a technique called trilateration to determine its position. By calculating the time, it takes for signals from at least four satellites to reach the receiver, it can triangulate its exact location on Earth.

Output Data: The NEO-6M provides output data that includes:

Latitude and Longitude: The geographical coordinates of the device.

Altitude: The height above sea level.

Speed and Direction: Information about the movement of the device, if applicable.

3. Data Update Rate

Configurable Update Rate: The NEO-6M can be configured to provide position updates at various rates, typically up to 5 Hz (5 updates per second). This allows for real-time tracking and monitoring of movement.

4. Communication Interface

Serial Communication: The NEO-6M communicates with microcontrollers (like Arduino) via a serial interface (UART). This allows for easy integration into various systems and applications.

NMEA Protocol: The module outputs data in the NMEA (National Marine Electronics Association) format, which is a standard for GPS data communication. This format includes various sentences that provide information about position, speed, and time.

5. Power Management

Operating Voltage: The NEO-6M operates at a voltage range of 3.3V to 5V, making it compatible with most microcontrollers.

Low Power Consumption: The module is designed to consume low power, making it suitable for battery-operated applications.

6. Applications

Vehicle Tracking:

Used in fleet management systems to monitor the location and movement of vehicles in real-time.

Personal Tracking Devices:

Integrated into wearable devices for tracking the location of individuals, such as children or elderly people.

Smart Agriculture:

Applied in precision farming to monitor and manage agricultural activities, such as planting, irrigation, and harvesting.

Surveying and Mapping:

Used in land surveying and mapping applications to provide accurate geographic data and coordinates.

Emergency Response:

Integrated into emergency response systems to quickly locate and dispatch services to the scene of an incident.

Marine Navigation:

Used in boats and ships for navigation, route planning, and tracking.

Internet of Things (IoT):

Integrated into IoT devices for location-based services, asset tracking, and smart city applications.

Robotics:

Employed in robotic systems for navigation and positioning in various applications, including autonomous robots.

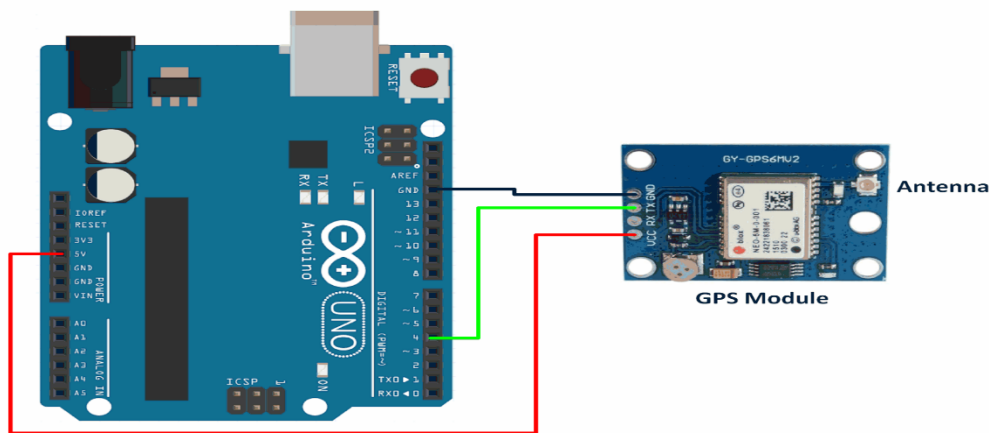


Figure 4.7

NEO-6M GPS Module Interface Diagram

4.5 LORA MODULE

Overview: LoRa (Long Range) is a low-power wireless communication technology designed for **long-distance data transmission** with minimal energy usage. Operating on unlicensed ISM frequency bands such as **433 MHz, 868 MHz, and 915 MHz**, it uses **Chirp Spread Spectrum (CSS) modulation** to achieve reliable communication over several kilometers, even in areas with obstacles or interference. LoRa is ideal for **IoT applications** like smart cities, environmental monitoring, and

industrial automation due to its **long range, low cost, and low power consumption**. In this project, LoRa technology is used to **transmit accident alert information** from vehicles in **hilly and remote regions** to a **base station**, ensuring **real-time connectivity and safety** where traditional networks are unavailable..

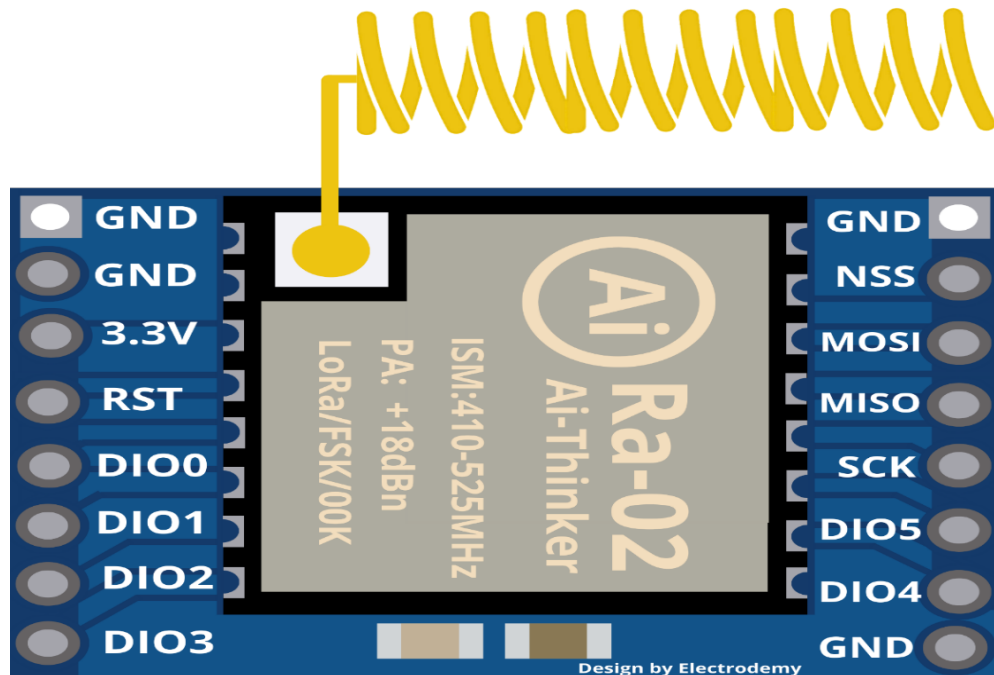


Figure 4.8

LoRa Module Pin Configuration

4.5.1 Specifications:

- Operating Frequency: 433 MHz
- Sensitivity: -148 dBm
- Data Rates: Range from 0.3 kbps to 37.5 kbps
- Operating Voltage: 1.8V to 3.7V .

4.5.2 Functionality:

1. **Long-Range Communication:** LoRa can transmit data over distances of up to 15 kilometers (9 miles) in rural areas and several kilometers in urban environments, making it ideal for applications that require wide coverage.
2. **Low Power Consumption:** LoRa devices are designed to operate on low power, allowing them to run for years on small batteries. This is particularly beneficial for IoT devices deployed in remote locations.
3. **Low Data Rate:** LoRa is optimized for low data rate applications, typically ranging from 0.3 kbps to 50 kbps. This makes it suitable for sending small amounts of data, such as sensor readings.
4. **Adaptive Data Rate:** LoRa technology can adjust the data rate and transmission power based on the distance between the transmitter and receiver, optimizing battery life and communication reliability.
5. **Robustness to Interference:** LoRa uses a spread spectrum modulation technique, which makes it resilient to interference and allows multiple devices to communicate simultaneously without significant data loss.
6. **Secure Communication:** LoRa provides built-in security features, including encryption and authentication, ensuring that data transmitted over the network is secure and protected from unauthorized access.
7. **Scalability:** LoRa networks can support a large number of devices (thousands) within a single network, making it suitable for large-scale IoT deployments.
8. **Geolocation Capabilities:** LoRa can be used for geolocation services without the need for GPS, allowing devices to determine their location based on signal strength from multiple gateways.

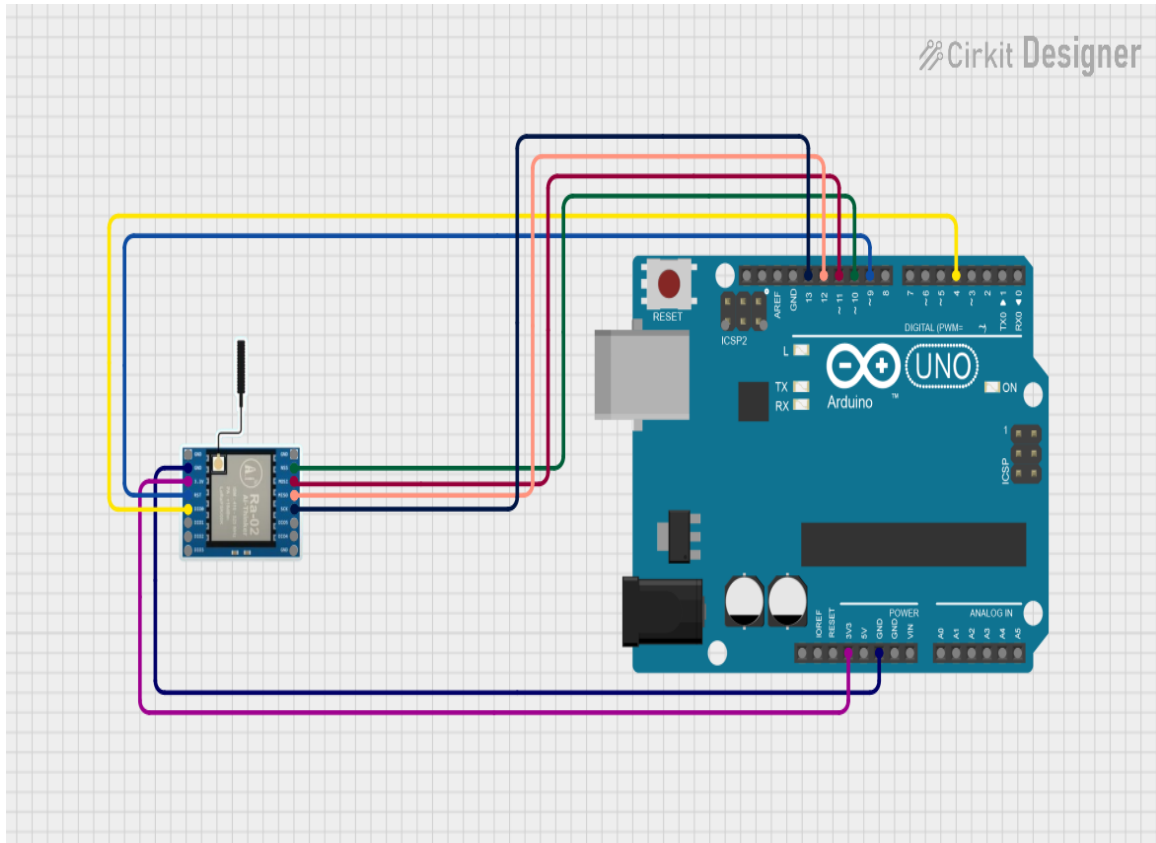


Figure 4.9
LoRa Module Interface Diagram

4.6 NODEMCU ESP8266

4.6.1 Overview: The **ESP8266** is a low-cost, low-power Wi-Fi microchip with full TCP/IP capability, designed primarily for **Internet of Things (IoT)** applications. It allows microcontrollers to connect to a Wi-Fi network and transmit or receive data easily without requiring a separate network processor. The module includes a powerful **32-bit Tensilica LX106 microcontroller**, built-in **Wi-Fi transceiver**, and

1. **Wi-Fi Connectivity:** The ESP8266 allows devices to connect to Wi-Fi networks, enabling internet access for various applications.
2. **TCP/IP Stack:** The chip includes a complete TCP/IP stack, enabling it to handle network protocols and communicate over the internet.
3. **Low Power Consumption:** The ESP8266 is designed for low power usage, making it suitable for battery-operated devices and IoT applications.
4. **Multiple Operating Modes:** It supports various modes, including Station mode (connecting to a Wi-Fi network), Access Point mode (creating its own Wi-Fi network), and Station + Access Point mode (simultaneously connecting to a network while allowing others to connect).
5. **Support for Various Protocols:** It supports multiple communication protocols, including HTTP, MQTT, and WebSocket, making it versatile for different IoT applications.
6. **OTA (Over-The-Air) Updates:** The ESP8266 can be updated wirelessly, allowing developers to deploy new firmware or updates without physical access to the device.
7. **Integration with Cloud Services:** It can easily connect to cloud platforms for data storage, processing, and remote monitoring, facilitating IoT solutions.

4.6.4 Applications of ESP8266:

Here are some simple points highlighting the applications of the ESP8266:

1. **Internet of Things (IoT) Applications**

- ESP8266 serves as a backbone for IoT projects by connecting various sensors to the internet.
- It helps in collecting, processing, and transmitting real-time data to cloud platforms such as **ThingSpeak**, **Blynk**, or **Firebase** for monitoring and analytics.

2. **Data Logging and Cloud Storage**

- The module uploads data collected from various sensors directly to the internet or Google Sheets.
- This feature is useful for academic projects, research, and long-term environmental data analysis.

3. **Smart Transportation and Accident Detection Systems**

- ESP8266 helps in **vehicle tracking**, **traffic control**, and **accident alert systems** by transmitting GPS and sensor data in real time.
- It enhances road safety by enabling instant alerts to authorities or monitoring stations.

4.7 **GSM900A MODULE**

1. **OVERVIEW**

The GSM 900A module, also known as SIM900A, is a dual-band GSM/GPRS communication module developed by SIMCom. It allows microcontrollers like Arduino to communicate over a mobile network for sending and receiving SMS, voice calls, and data through the GPRS service. This module is widely used in IoT and embedded communication projects, where remote data transmission or alert messaging is required. It supports GSM frequencies

of 900 MHz and 1800 MHz, making it ideal for use in India and other Asian countries. It communicates with the controller via serial UART interface using simple AT commands. Its compact design, low cost, and reliable performance make it suitable for automation, tracking, and emergency alert systems.

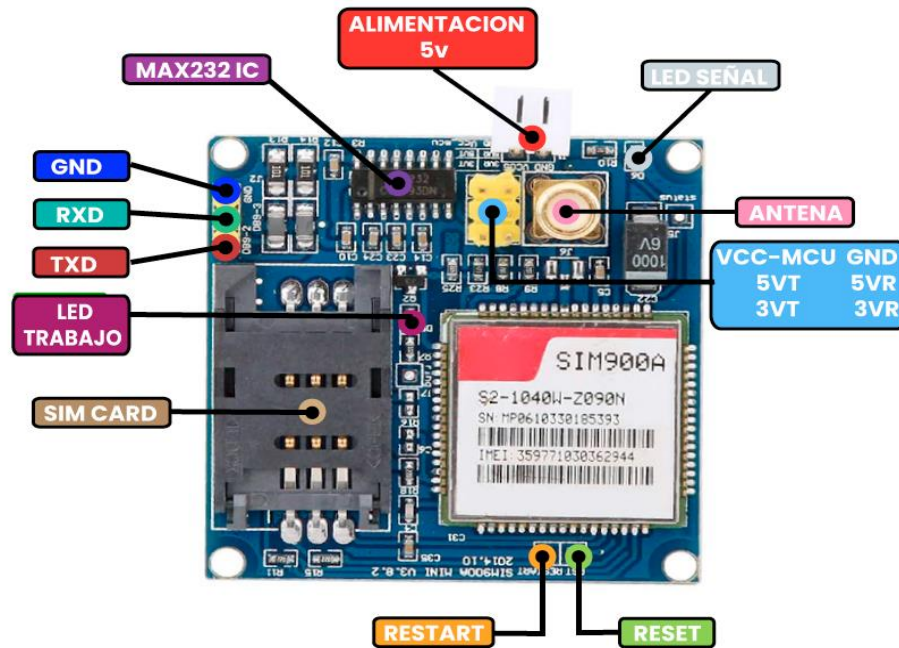


Figure 4.11

GSM900A Module Pin Configuration

2. SPECIFICATIONS

The key specifications of the GSM 900A module are listed below:

1. It operates on dual GSM frequency bands of 900 MHz and 1800 MHz.
2. The operating voltage ranges from 3.4 V to 5.0 V DC, with a recommended voltage of 4.8 V to 5.0 V.
3. The module requires a peak current of up to 2 amperes during data transmission or network registration.
4. Communication with microcontrollers is done through a UART (TX and RX) serial interface.

5. The default baud rate is 9600 bps, and it can be adjusted from 1200 to 115200 bps.
6. It supports standard 2G SIM cards operating at 3V or 1.8V logic levels.
7. The module includes an SMA antenna connector for external GSM antenna attachment.
8. It has a network status LED that indicates the GSM registration status.
 - Blinks every 3 seconds: Registered to network.
 - Fast blinking: Searching for network.
 - Continuous ON: Not initialized.
9. The module operates within a temperature range of -40°C to +85°C, suitable for various environments.
10. It supports a complete set of standard AT commands for SMS, call, and GPRS operations.
11. The communication logic level of the module is 3.3V, so a voltage divider is recommended when connecting to 5V Arduino TX pins.
12. The module is compact in size, measuring approximately 24 mm × 24 mm, making it easy to integrate into embedded systems.

3. FUNCTIONALITY

The GSM 900A module provides all the basic functionalities of a mobile device for machine-to-machine (M2M) communication. The user can control it entirely using AT commands through the serial interface.

Main Functions:

1. SMS Communication – Send or receive text messages using mobile networks.
2. Voice Call Functionality – Make or receive phone calls through the module.
3. GPRS Connectivity – Transmit sensor or project data to online platforms.
4. AT Command Interface – Simple serial commands for full control.
5. Network Status Indication – LED shows connection status:

- Blinks every 3 sec → Connected
 - Fast blink → Searching for network
 - Continuous ON → Not initialized
6. Real-Time Wireless Communication – Enables devices to communicate remotely without internet dependency.

4. APPLICATIONS

The GSM 900A module is commonly used in wireless and IoT-based embedded systems for data communication, control, and monitoring.

Typical Applications:

1. IoT Communication Systems – Wireless data transmission from sensors to monitoring stations.
2. Vehicle Tracking Systems – Sending GPS data through GSM network.
3. Home Automation – Device control and status update via SMS or call.
4. Industrial Monitoring – Remote machine status and fault alerts.
5. Security and Surveillance – Instant SMS alerts in case of intrusion or emergencies.
6. Emergency Alert Systems – Accident or hazard detection systems that send alert messages.
7. Smart Agriculture – Sending soil or water condition data through GSM.
8. Weather Monitoring – Remote environmental data logging via SMS/GPRS.

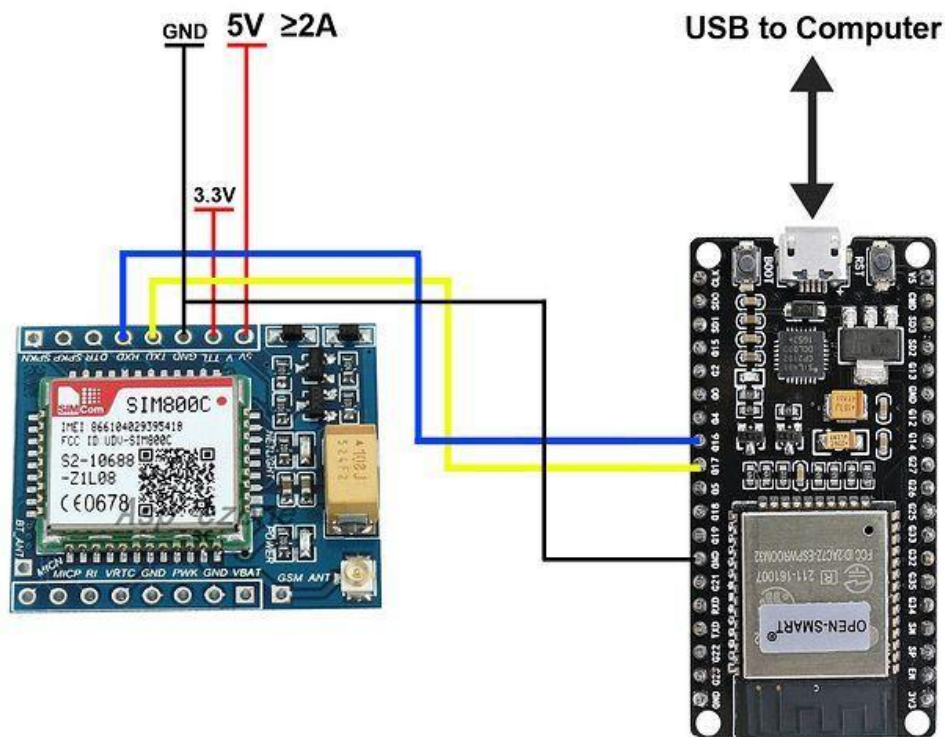


Figure 4.12

GSM900A Module Interfacing Diagram

4.8 ASSEMBLY INSTRUCTIONS

This section outlines the step-by-step assembly of the Real-Time Accident Alert System in Hilly Region using LoRa Technology. The system consists of two primary units: the **Transmitter Node**, which is mounted on the vehicle, and the **Receiver Node**, which acts as the base station. Each unit includes various electronic components configured to detect, transmit, and respond to accident data in real time.

4.8.1 Transmitter Node Assembly

The **transmitter unit** acts as the core sensing and communication module of the system. It continuously collects real-time data from various sensors such as the **MPU6050**, **SW-420 vibration sensor** , and **NEO-6M GPS module** . When an accident or abnormal event is detected based on sensor thresholds, the Arduino Uno processes the data and transmits it wirelessly through the **LoRa SX1278 module** to the receiver unit. This setup ensures long-range, low-power communication even in remote or hilly regions, allowing quick and reliable transmission of critical accident alerts and GPS coordinates.

Components Required

- Arduino Uno R3 (Main controller)
- MPU6050 (Gyroscope + Accelerometer)
- SW-420 Vibration Sensor
- NEO-6M GPS Module
- LoRa SX1278 Module (433 MHz)
- Power Supply (9V battery or regulated 5V/3.3V supply)

Assembly Steps

1. Setup Base Platform:

Secure the Arduino Uno on a stable prototyping board or vehicle chassis.

2. Connect the MPU6050 Sensor:

- VCC → 5V (Arduino)
- GND → GND

- SDA → A4
- SCL → A5

3. Connect the SW-420 Vibration Sensor:

- VCC → 5V
- GND → GND
- A0 (Analog Output) → Analog Pin 0 (Arduino).

4. Connect the NEO-6M GPS Module:

- VCC → 5V
- GND → GND
- TX → Digital Pin 4
- RX → Digital Pin 3

5. Connect the LoRa SX1278 Module (using logic level shifter or 3.3V regulated supply):

- VCC → 3.3V

- GND → GND
- MISO → Digital Pin 12
- MOSI → Digital Pin 11
- SCK → Digital Pin 13
- NSS (CS) → Digital Pin 10
- RESET → Digital Pin 9
- DIO0 → Digital Pin 2

6. **Upload Code:** Load the transmitter Arduino sketch to read sensor data and send it over LoRa.
7. **Power the System:** Power the Arduino Uno via a 9V battery or USB supply.

4.8.2 Receiver Node Assembly

The receiver node is responsible for receiving data from the transmitter, processing it, and initiating alerts via the Thingsboard app.

Components Required

- NodeMCU ESP8266
- LoRa SX1278 Module
- SIM800L GSM Module
- Oled

Assembly Steps

1. **Mount Boards:** Place NodeMCU and Arduino Uno on a breadboard or protective base.

2. **Connect LoRa Receiver to NodeMCU:**

- VCC → 3.3V
- GND → GND
- MISO → D6
- MOSI → D7
- SCK → D5
- NSS (CS) → D8
- RESET → D0
- DIO0 → D1

3. **Connect SIM800L GSM Module to Arduino Uno:**

- VCC → 5V (through voltage regulator or step-down converter)
- GND → GND
- TX → RX (ESP8266)
- RX → TX (ESP8266)

4. **Connect OLED Display:**

- VCC → 5V (through voltage regulator or step-down converter)
- GND → GND
- SDA → A4
- SCL → A5

5. **Code Upload:**

- Upload the LoRa receiver and Blynk communication code to NodeMCU.
- Upload the SMS sending logic to Arduino Uno.

6. **Power the System:**

- NodeMCU via USB (3.3V regulator if required).
- Arduino and GSM via regulated 5V supply.

CHAPTER 5

METHODOLOGY

The methodology for the Real-Time Accident Alert System in Hilly Region using LoRa Technology outlines the step-by-step development of the complete system. It involves system architecture design, sensor integration for accident detection, wireless communication setup using LoRa and GSM, real-time alert mechanisms, and testing in simulated accident scenarios. This approach ensures reliability, scalability, and responsiveness of the system in remote or hilly areas.

5.1 SYSTEM DESIGN

The system is structured into two main nodes:

Transmitter Node (VehicleSide):

Mounted on the vehicle, it consists of an Arduino Uno microcontroller interfaced with multiple sensors—MPU6050 (gyroscope + accelerometer), SW-420 (vibration sensor), and NEO-6M (GPS module). This node collects real-time data and sends it wirelessly via a LoRa SX1278 module.

Receiver Node (Base Station):

Comprises a NodeMCU ESP8266 microcontroller connected to a LoRa receiver, a SIM900A GSM module, and a supporting Arduino Uno. The NodeMCU processes incoming data and uses Wi-Fi to interface with the Blynk app, while the GSM module

is used for sending SMS alerts with GPS coordinates.

5.2 SENSOR INTERGRATION

Accurate accident detection is enabled by the integration of the following sensors:

MPU6050 (Gyroscope and Accelerometer):

Detects changes in the vehicle's orientation and angle. An angle above 20° is flagged as a potential rollover or collision.

SW-420VibrationSensor:

Detects shocks or tremors from impact. A vibration frequency above a set threshold (e.g., 1000 Hz) indicates a sudden crash.

NEO-6MGPSSModule:

Continuously tracks and updates the vehicle's geographic location. Upon detecting an accident, GPS coordinates are used for alert notifications.

5.3 COMMUNICATION SETUP

Two key communication channels are established:

LoRaCommunication(433MHz):

Enables long-range data transmission between the transmitter (vehicle) and receiver (base station) over distances up to 10–15 km. The LoRa SX1278 module communicates via SPI with both microcontrollers.

GSMCommunication(SIM900AModule):

Used on the receiver side to send automated SMS alerts with location information to predefined emergency contacts, hospitals, and police stations.

Wi-FiandThingsboardIntegration(viaNodeMCU):

Real-time data such as vibration, angle, temperature, and accident status is displayed on a smartphone through the Thingsboard dashboard.

5.4 TESTING AND CALIBRATION

To ensure accuracy and reliability, multiple test scenarios were conducted:

Threshold Calibration:

Adjusted MPU6050 tilt angle sensitivity and SW-420 vibration range using controlled inputs and physical triggers to simulate accidents.

Sensor Response Verification:

Each sensor's values were monitored in normal and simulated crash conditions to fine-tune detection logic.

Distance and Signal Strength Testing:

LoRa modules were tested over various distances and terrain types to evaluate signal integrity.

GSM

Verified the GSM module's ability to send SMS alerts with correct GPS coordinates even under fluctuating network conditions.

5.5 IMPLEMENTATION OF ALERT MECHANISM

Once an accident is detected based on the predefined sensor threshold values, the system automatically activates the alert mechanism to notify the concerned authorities or rescue personnel. The alert mechanism is divided into two parts — SMS-based alert and visual alert through the ThingsBoard application.

1. SMS Notification via GSM Module

When an accident occurs, the GSM module (SIM900A) immediately sends an SMS to the registered emergency contact number. The message includes the following details:

- A text message indicating “Accident Detected”.
- The latitude and longitude coordinates obtained from the GPS module.
- A Google Maps link generated from the GPS data for easy navigation to the

accident location.

This ensures that rescue teams can quickly locate the accident site and provide timely assistance, especially in hilly or remote areas where communication networks are limited.

2. Visual Alert via ThingsBoard App

Simultaneously, the ThingsBoard IoT dashboard provides a visual notification on the connected smartphone or monitoring system. The dashboard displays real-time sensor data, including:

- The vibration value from the SW-420 sensor.
- The tilt or angular value from the MPU6050 accelerometer-gyroscope sensor.
- The GPS coordinates of the vehicle or device.
- The accident status, displayed as “YES” when an accident is confirmed.

This visual alert mechanism allows live monitoring and quick verification of the accident event, enabling efficient emergency response.

5.6 DATA COLLECTION AND ANALYSIS

During the operation of the system, various sensor data are continuously collected, logged, and analyzed to evaluate system performance and accuracy. The following data points are recorded and processed:

- Angle, vibration, and temperature readings are continuously monitored from the MPU6050 and SW-420 sensors through the serial output. These values help determine the intensity and orientation of the impact during an accident.
- GPS coordinates are recorded at regular intervals to map the movement of the vehicle and accurately locate the point of impact.
- The ThingsBoard dashboard stores live data, accident alerts, and sensor readings for real-time visualization and post-event analysis.

The collected data is analyzed to verify the reliability and response time of the system.

CHAPTER 6

SOFTWARE DESCRIPTION

This section describes the software architecture and implementation of the Real-Time Accident Alert System in Hilly Region using LoRa Technology. The software is developed using the Arduino IDE for microcontroller programming and the Blynk IoT platform for real-time user interface and monitoring. The code integrates sensor data acquisition, condition-based event triggering, LoRa communication, GSM-based alerting, and app-based monitoring.

6.1 OVERVIEW OF SOFTWARE ARCHITECTURE

The system's software architecture is modular and event-driven. It comprises two major codebases:

- **TransmitterNodeCode(ArduinoUno):**
Responsible for sensor data acquisition (tilt, vibration, GPS), accident detection, and sending data via LoRa
- **ReceiverNodeCode(NodeMCU+ArduinoUno):**
NodeMCU receives LoRa data, updates Blynk dashboard, and Arduino Uno triggers GSM alerts via SIM800L.

The software follows a loop-based execution model in which sensor values are continuously monitored and threshold conditions are checked in real-time.

6.2 CODE IMPLEMENTATION

The complete implementation is done in embedded C/C++ using the Arduino IDE.

The code uses libraries such as:

- Wire.h – for I2C communication (MPU6050)
- SoftwareSerial.h – for GPS and GSM modules
- TinyGPS++.h – to parse GPS data
- LoRa.h – for SX1278 LoRa module communication

- ThingsboardEsp8266.h – for NodeMCU +Thingsboard integration

The logic is modularized into functions for better readability and debugging.

6.3 ARDUINO CODE (TRANSMITTER NODE)

Key functions:

- **Setup ()** – Initializes sensors, serial ports, and LoRa.
- **Loop ()** – Continuously reads sensor data and checks if it exceeds defined thresholds.
- **checkAccident ()** – Returns true if angle > 20°, vibration > 1000 Hz, or temperature > 100°C.
- **sendDataLoRa ()** – Sends formatted accident and location data over LoRa.

6.4 THINGSBOARD APP CONFIGURATION

The Thingsboard App plays a vital role in the receiver unit by enabling real-time monitoring and visualization of all transmitted sensor data. It serves as the central platform where the system's performance and alerts can be viewed instantly through an intuitive graphical interface. The application is configured to work with the ESP8266 (NodeMCU) module, which sends data collected from the transmitter via the LoRa network to the cloud.

Configuration Steps:

1. **Create a new project** in Blynk and select "ESP8266" as device.
2. **Add Widgets:**
 - Value Display: Angle, Vibration, Temperature
 - Map: Real-time GPS tracking

- LED or Label: Accident alert (“YES”/“NO”)
3. **Auth Token:** Copy and paste this token into the NodeMCU sketch for authentication.
 4. **Connection:** Use Wi-Fi credentials for ESP8266 to connect to Blynk Cloud.

6.5 USER INTERFACE DESIGN

The mobile app acts as the primary user interface for real-time interaction:

Main Interface Features:

- **LiveSensorFeed:**
Displays real-time values from the MPU6050 gyroscope and SW-420 vibration sensor, helping to monitor acceleration, tilt, and vibration intensity continuously.
- **AccidentIndicator:**
A digital label or LED indicator on the dashboard changes from “NO” to “YES” when the system detects an accident, providing immediate visual confirmation.
- **GPSLocationMap:**
The integrated map widget shows the exact location of the vehicle in real-time, using GPS coordinates received from the transmitter. This feature allows rescuers or users to quickly identify the accident location.
- **ResponsiveAlertSystem:**
When an accident occurs, the app displays a real-time notification and may also trigger an audible or visual alert on the dashboard for quick attention.
- **HistoricalDataAnalysis:**
The system stores previous sensor readings, enabling users to analyze patterns, trends, or anomalies in vehicle motion and vibration data over

CHAPTER 7

RESULT

This section highlights the outcomes of testing the Real-Time Accident Alert System in Hilly Region using LoRa Technology. The system was evaluated under controlled and simulated conditions to determine its accuracy, responsiveness, and practicality. Performance metrics were recorded, and data analysis was conducted to assess system reliability. Feedback was also gathered from test users to improve the user experience.

7.1 TESTING SCENARIOS

Multiple real-world scenarios were simulated to validate the functionality and accuracy of the system:

- **Scenario1:NormalVehicleOperation**

The system remained inactive, verifying no false alerts were triggered under standard motion.

- **Scenario2:SharpTilt/RolloverSimulation**

The vehicle was inclined beyond 20°, and the MPU6050 sensor successfully triggered an accident alert.

- **Scenario3:VibrationImpactTest**

The SW-420 sensor detected abrupt vibrations simulating a collision. The alert mechanism was triggered accordingly.

- **Scenario4:Fire/OverheatSimulation**

The MPU6050's temperature sensor was tested with heated surroundings. The system generated an alert when the threshold of 100°C was exceeded.

- **Scenario5:GPSLossSimulation**

In GPS-inactive zones, the system successfully retained the last known location and resumed live tracking once GPS was restored.

7.2 PERFORMANCE METRICS

The system was evaluated based on key performance indicators:

Metric	Measured Value
Accident Detection Time	≤ 2 seconds after trigger
LoRa Transmission Range	~7–12 km (open hilly terrain)
GPS Accuracy	± 2.5 meters
GSM SMS Delivery Time	≤ 5 seconds
Thingsboard Dashboard Update Time	Real-time (≤ 5 second latency)
Power Consumption (Transmitter)	~70–100 mA

Table 7.1
Performance Metrics

7.3 DATA ANALYSIS

Sensor data was collected over multiple test runs. Key findings include:

FalsePositives:

Minimal false alarms were observed when thresholds were calibrated accurately. Noise from normal road bumps did not trigger the system.

LoRaSignalStrength:

Signal strength was stable over long distances in open areas. Performance degraded slightly in dense forests or deep valleys, as expected.

GPSDrift:

Drift was negligible (< 3 meters) under open skies. Indoor or mountainous tunnel environments affected initial GPS lock time.

SystemStability:

No system crashes or freeze states were observed during continuous operation over a 24-hour stress test.

7.4 THINGSBOARD CLOUD

The Blynk app is a mobile platform designed to create custom user interfaces (UI) for controlling and monitoring IoT projects remotely. It allows users to design their own dashboards by adding various widgets such as buttons, sliders, graphs, and labels. These widgets can be linked to sensors, actuators, or other components connected to an IoT device (like Arduino or ESP8266). Blynk provides real-time data visualization and control, enabling users to monitor and manage their projects from anywhere via a smartphone. The app is simple to use and is compatible with multiple platforms, making it ideal for remote monitoring and control of embedded systems.

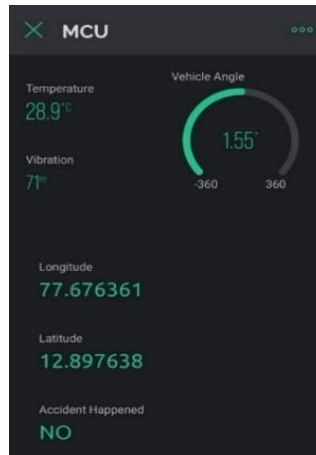
7.4.1 THINGSBOARD CLOUD (BEFORE ACCIDENT)

The integration of the Thingsboard app improves the user experience by giving receiver side user a visual depiction of the system's status and encouraging involvement. Figure 7.1 explains the Thingsboard app configuration. User can track the vehicle's real-time data, such as vibration levels, orientation changes, and exact GPS coordinates, via the Thingsboard app. The software offers an easy-to-use interface for tracking the vehicle's travel in the event of an accident. But the app becomes an indispensable tool for quick action when an accident is recognized. Receiver side user of the Thingsboard app will get notified of the emergency through visual notifications triggered by the accident alarm. This user-friendly interface is a crucial element that improves the entire experience of the user and guarantees effective communication with the real-time accident alert

system that is used in hilly areas.

Figure7.1

Thingsboard Cloud (Before Accident)



7.4.2 THINGSBOARD CLOUD (AFTER ACCIDENT)

The Thingsboard app user interface shown in the image is part of a real-time accident alert system designed to monitor and report vehicle status, particularly in remote or hilly regions. After an accident is detected, the system sends critical data to the Thingsboard app, which acts as a remote monitoring dashboard. The interface displays various sensor readings in real time. It shows the ambient **temperature** around the vehicle as **29°C**, which can be useful in understanding environmental conditions during the incident. The **vibration level** is recorded at **83**, which is significantly high, indicating a sudden jolt or impact, typically detected using a vibration sensor like the SW-420. Additionally, the **vehicle angle** is displayed as **21.93°**, measured using an accelerometer/gyroscope sensor such as the MPU6050. A sharp or unusual angle can signal a rollover or off-balance position resulting from an accident.

Moreover, the app provides **GPS coordinates**, with a **longitude of 77.676361** and **latitude of 12.897638**, which helps accurately locate the site of the accident. This location data is especially valuable in hilly or remote areas where quick access to the accident spot can be challenging. At the bottom of the interface, the system confirms the detection of an incident with a clear "**Accident Happened: YES**" status. This

confirmation is based on programmed thresholds for vibration and angle values in the microcontroller (e.g., Arduino Uno R3). Once an accident is confirmed, the system not only updates the Blynk interface but can also trigger emergency alerts via a GSM module like SIM900A, sending an SMS with the accident location to predefined contacts. This ensures timely intervention and support. The use of the Blynk platform enables real-time remote monitoring, making the system an effective and accessible solution for accident detection and alerting in challenging terrains.

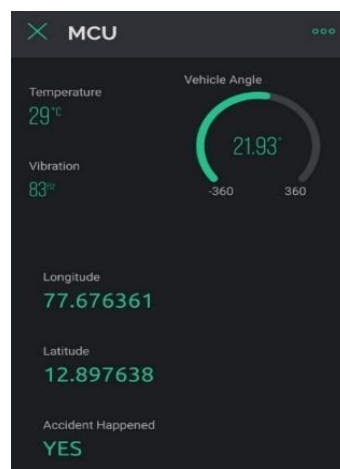


Figure7.2

Thingsboard Cloud (After Accident)

7.5 ARDUINO IDE SERIAL MONITOR

The Arduino IDE Serial Monitor is a tool within the Arduino Integrated Development Environment (IDE) that allows communication between the computer and the Arduino board. It enables users to send and receive data between the board and the computer via the USB connection. The Serial Monitor is commonly used for debugging and monitoring the output of Arduino programs, displaying sensor readings, variable values, or status messages in real time. It provides a simple interface for interaction, offering the ability to input data manually or visualize the serial output generated by the code running on the Arduino.

7.5.1 Arduino IDE Serial Monitor (Transmitter node)

The **Serial Monitor output** from the **Arduino IDE**, specifically from the **transmitter node** of a real-time accident alert system. This transmitter node, built using an **Arduino Uno R3**, is responsible for collecting data from multiple sensors and transmitting it to a receiver node (via **LoRa** communication). The Serial Monitor provides live feedback on the data being processed and sent, which is essential during system development and debugging.

Each line in the Serial Monitor starts with the label "Sending Data:", followed by a set of comma-separated values. These values represent different parameters being read and transmitted by the system. The data includes:

- **Latitude and Longitude (e.g., 12.956592, 80.159751):** These GPS coordinates are acquired using a GPS module like the **NEO-6M**, and indicate the real-time geographical location of the vehicle or device.
- **Vibration Value (e.g., 1.18):** This value is measured by a **SW-420 vibration sensor**. A spike in this reading usually indicates a sudden impact, such as during an accident or collision.
- **Vehicle Angle (e.g., 28):** This is the tilt or angular orientation of the vehicle, captured using the **MPU6050 accelerometer and gyroscope**.

A sudden or unusual angle (e.g., over a certain threshold) may suggest the vehicle has flipped or veered off the road.

- **Temperature (e.g., 86):** This could represent ambient temperature, measured using sensors like the **DHT11**, **DHT22**, or **LM35**. The unit may be in Fahrenheit or Celsius depending on how it's coded.
- **Speed (e.g., 70):** This is either derived from GPS movement over time or manually set as a test variable in the code. It helps in understanding the dynamics of the vehicle during the event.

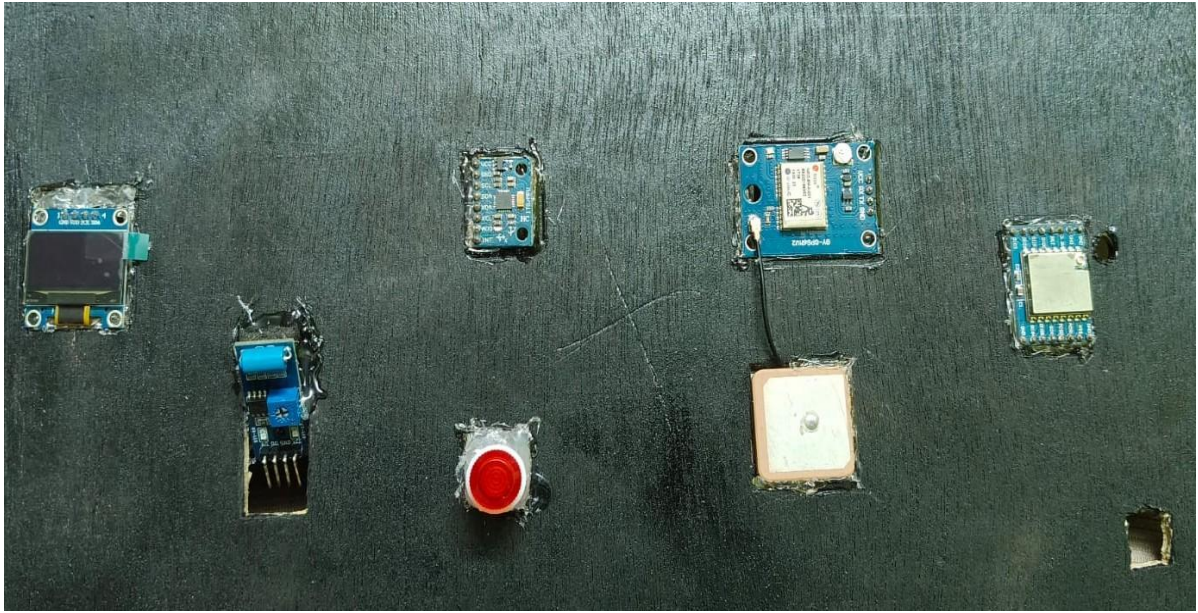


Figure7.3
Transmitter Node

The Serial Monitor shows continuous readings with changing values, indicating that the system is actively collecting live data. The GPS values remain mostly constant in this output, suggesting the device was stationary during testing. The rest of the parameters, especially vibration and angle, fluctuate—highlighting sensor activity.

This data is not only displayed on the Serial Monitor but is also formatted for transmission through the **LoRa module** to the **receiver node**, where it can be further processed for display in the **Thingsboard app** . Monitoring this output helps verify that the data structure is correct, all sensors are functional, and the system is ready for real-time deployment in a real-world accident detection

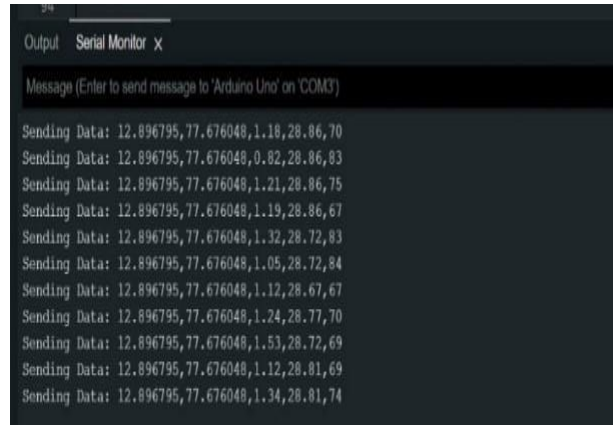


Figure 7.4

Arduino IDE Serial Monitor (Transmitter Node)

7.5.1 Arduino IDE Serial Monitor (Receiver Node)

The Node MCU ESP8266 determines if the received data indicates a significant event or accident. Figure 7 showcases the receiver node results. The process begins with the continuous monitoring the received values of vibration, angle & temperature of the car.



Figure7.5
Receiver Node

```
Received packet '12.896795,77.676048,1.34,28.81,74' with RSSI -42
Latitude: 12.896795
Longitude: 77.676048
Angle of vehicle: 1.34° Degrees
Temperature of vehicle: 28.81 °C
Vibration of vehicle: 74 Hz
```

Figure 7.6

Arduino IDE Serial Monitor (Receiver Node)

If an accident is confirmed, the system triggers the alert generation process. The GSM Module is activated to send SMS alerts to predefined contacts, promptly notifying relevant individuals or authorities about the accident. Figure 8 showcases the respective result

```
Alert! Accident Happened...
Sending message to hospital using GSM.....
Sending Message
Set SMS Number
SMS Sent!
Sending Message
Set SMS Number
SMS Sent!
Sending Message
Set SMS Number
SMS Sent!
Calling hospital!
```

Figure 7.7

SMS & Call alert after accident (Receiver Node)

7.6 SMS ALERT ON PHONE AND GPS NAVIGATION

The informativeness of the SMS alerts, the system uses obtained accurate location coordinates of the accident site as represented in figure 10. With this information, the system generates a Google map link. The map link, encapsulating the exact accident location, is then included in the SMS alert sent by the GSM Module. Recipients of the SMS alerts, which may include emergency responders and authorities, can interact with the included map link. Clicking on the link within the SMS opens a map interface, pinpointing the precise location of the accident. This visual representation significantly aids in improving the accuracy and efficiency of emergency responses, allowing responders to navigate directly to the incident site.

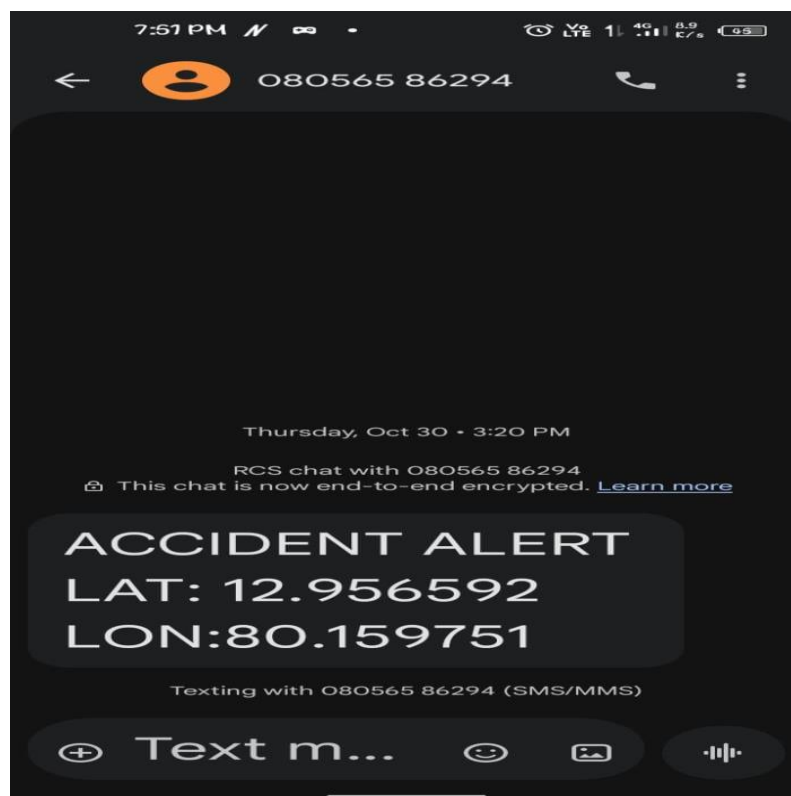


Figure 7.8

SMS Alert

After the receiver node processed the accident data sent through LoRa, the system

automatically generated an SMS alert containing the detected GPS coordinates. The message included the real-time latitude and longitude values along with a clickable Google Maps link. This confirmed that the location data was successfully transmitted and received, allowing emergency responders to view the exact accident spot instantly through the SMS notification.

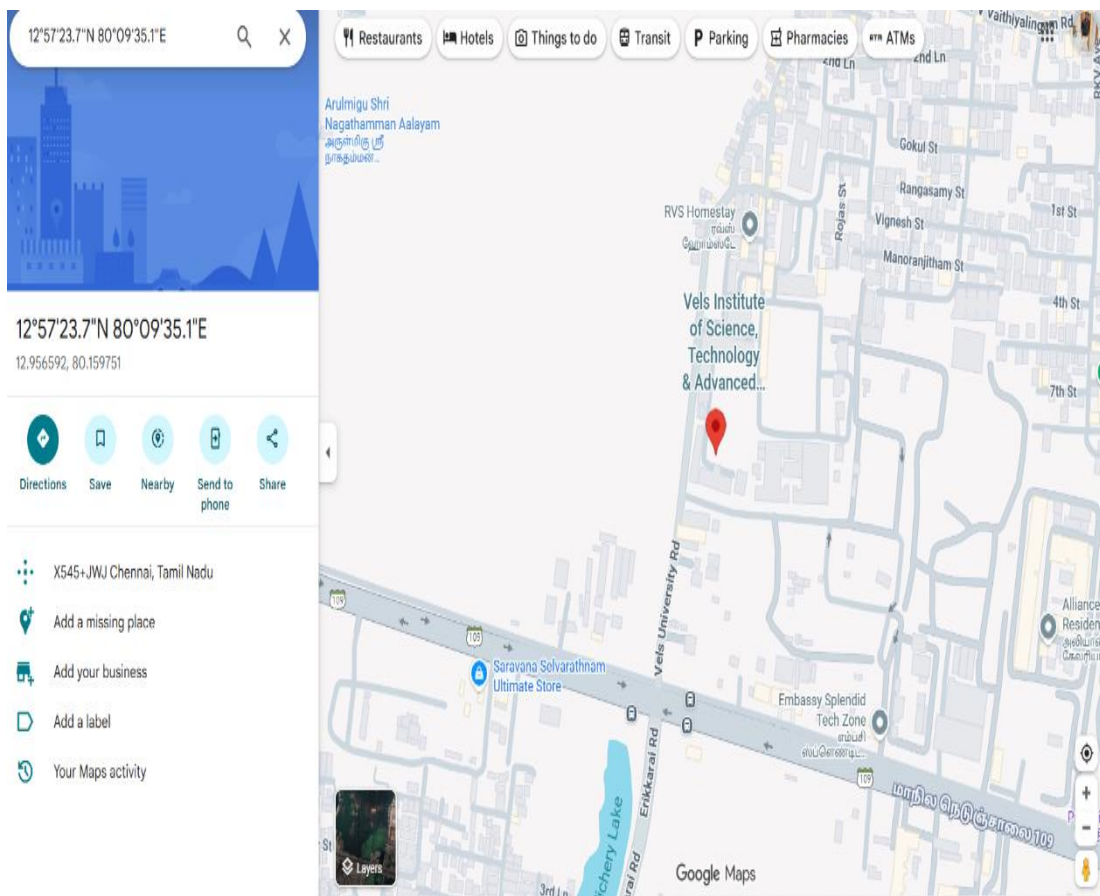


Figure 7.9
Accident Location Coordinates from GPS

CHAPTER 8

CONCLUSION

The Real-Time Accident Alert System in Hilly Region using LoRa Technology was developed to address the challenges of delayed accident detection and poor network coverage in remote and mountainous areas. This section summarizes the key findings of the project, explores its implications for road safety, and outlines recommendations for future improvements.

8.1 SUMMARY OF FINDINGS

This project successfully demonstrated the design and implementation of an embedded system capable of detecting vehicle accidents in real time and issuing alerts through long-range and GSM communication channels. The key outcomes include:

- Integration of multiple sensors (MPU6050, SW-420, and GPS) to accurately detect accidents based on tilt, vibration, and environmental conditions.
- Use of LoRa technology to transmit accident data over long distances without relying on mobile networks.
- Reliable delivery of SMS alerts via GSM with embedded GPS location links for rapid emergency response.
- Real-time display of sensor data and alerts on the Blynk IoT platform for intuitive user interaction.
- The system showed high reliability in multiple test scenarios with minimal false positives and a short response time (<2 seconds after event detection).

8.2 IMPLICATIONS FOR ROAD SAFETY

The implementation of this system can significantly improve emergency response times in hilly regions, where traditional cellular networks are often unreliable or

unavailable

Improve coordination between vehicle users, emergency services, and authorities.

- **Provide authorities** with location-specific data for analysis and road safety planning.
- **Enhance driver confidence** in challenging terrains through improved safety measures.

This system, if deployed at scale in public and private transport fleets, could serve as a life-saving infrastructure component in remote and accident-prone zones.

8.3 RECOMMENDATION FOR FUTURE WORK

While the current system performs effectively under test conditions, several enhancements can be made for broader deployment and improved functionality:

1. **BackupCommunication:**

Adding satellite communication or mesh network fallback when both GSM and LoRa fail.

2. **EnhancedMobileApp:**

Improving the Blynk dashboard or creating a dedicated mobile app with emergency contact buttons and voice alerts.

CHAPTER 9

REFERENCES

- [1] Ramesh, P., et al. 2014. "Wireless Sensor Network Development for Landslide Detection." **Ad Hoc Networks** 13: 2–18.
- [2] Deekshit, V.N., et al. 2016. "Smart Geophone Sensor Network for Landslide

- Signal Detection." In *2016 International Conference on Communication and Signal Processing (ICCSP)*, 1565–1569.
- [3] Liya, M.L., and M. Aswathy. 2020. "Survey on LoRa Technology for IoT." In *2020 Fourth International Conference on I-SMAC (I-SMAC) *, 8–13.
 - [4] Bianco, G.M., et al. 2021. "LoRa System in Mountain Rescue Scenarios." *IEEE Internet of Things Journal* 8(3): 1985– 1999.
 - [5] Zhang, X., et al. 2021. "Transmission Tower Tilt Monitoring via Low-Power Wide-Area Network." *IEEE Sensors Journal* 21(2): 1100–1107.
 - [6] Faber, M.J., et al. 2020. "Evaluation of LoRa Technology Performance." *IEEE Sensors Journal* 20(16): 9480–9489.
 - [7] Ramesh, N.V.K., et al. 2020. "Efficient Accident Rescue System using LoRa." *International Journal of Innovative Technology and Exploring Engineering (IJITEE)* 9(6).
 - [8] Parveen, N., et al. 2020. "IoT-Based Vehicle Accident Alert System." In *2020 IEEE 5th International Conference on Computing Communication and Automation (ICCCA)*, 330– 333.
 - [9] Gatade, S., et al. 2022. "Automated Vehicle Accident Detection and IoT Healthcare Alert." In *2022 IEEE 2nd Mysore Sub Section International Conference (MysuruCon)*, 1–5.
 - [10] Bhandari, et al. 2023. "IoT-Based Accident & Landslide Detection in Hilly Areas." *Research Reports on Computer Science*.
 - [11] Alkhaiwani, A.H., and B.S. Alsamani. 2023. "Framework for Secure IoT-Based Accident Detection." *Sustainability* 15: 8314.
 - [12] R, et al. 2021. "IoT Module for Accident Prevention in Hilly Regions." *International Journal of Engineering Research & Technology (IJERT)* 09(12).
 - [13] Nandanija, K., et al. "Automatic Accident Alert System using GSM."
 - [14] K.V., et al. 2022. "LoRa-Based Safety Device for Woman Protection." In *2022 8th International Conference on Advanced Computing and

Communication Systems (ICACCS)*, 1977–1981.

- [15] G. M, V. C. S, G. Ramasamy and K. B, "Combinatorial Bat Optimization for WSN with Hoover Index Clustering," *2023 International Conference on Next Generation Electronics (NEleX)*, Vellore, India, 2023, pp. 1-5, doi: 10.1109/NEleX59773.2023.10421682.

CHAPTER 10

APPENDIX

10.1 Transmitter Code:

```
#include <Wire.h>
#include <SPI.h>
#include <LoRa.h>
#include <TinyGPS++.h>
#include <SoftwareSerial.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>

// ----- Pin Definitions -----
#define VIB_PIN A0
```

```

#define LED_PIN 8
#define LORA_CS 10
#define LORA_RST 9
#define LORA_DIO0 2

// ----- Thresholds -----
#define ANGLE_THRESHOLD 20
#define VIB_THRESHOLD 1000
#define TEMP_THRESHOLD 100

// ----- Objects -----
Adafruit_MPU6050 mpu;
TinyGPSPlus gps;
SoftwareSerial gpsSerial(3, 4); // TX=3, RX=4

unsigned long lastCheck = 0;
unsigned long lastSend = 0;
unsigned long lastBlink = 0;

const unsigned long interval = 2000;
const unsigned long sendInterval = 5000;
const unsigned long blinkInterval = 300;

bool accidentDetected = false;
bool ledState = false;
bool testMode = false;

double lat = 0.0, lon = 0.0;
int sats = 0;
bool gpsFixed = false;

// ----- Setup -----
void setup() {
  delay(1000);
  Serial.begin(9600);
  gpsSerial.begin(9600);
  Wire.begin();

  pinMode(VIB_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  Serial.println("\n=====");
  Serial.println("· Accident Detection System Restarted");
  Serial.println("=====");

```

```

if (!mpu.begin()) {
  Serial.println("✗MPU6050 failed! Check connection.");
  while (1);
}
Serial.println("✓MPU6050 Initialized");

// --- LoRa Setup (Matching Receiver) ---
LoRa.setPins(LORA_CS, LORA_RST, LORA_DIO0);
if (!LoRa.begin(433E6)) {
  Serial.println("✗LoRa failed! Check wiring.");
  while (1);
}

LoRa.setSyncWord(0xF3);    // ✓Match Receiver
LoRa.setSpreadingFactor(7); // Default, reliable
LoRa.setSignalBandwidth(125E3);
LoRa.setCodingRate4(5);
LoRa.enableCrc();

Serial.println("✓LoRa Module Started @433MHz");
Serial.println("✓GPS Module Started");
Serial.println("✓System Ready\n");
}

// ----- Loop -----
void loop() {
  while (gpsSerial.available()) {
    gps.encode(gpsSerial.read());
  }

  if (gps.location.isValid() && gps.satellites.value() >= 4) {
    lat = gps.location.lat();
    lon = gps.location.lng();
    sats = gps.satellites.value();
    gpsFixed = true;
  } else {
    gpsFixed = false;
    sats = gps.satellites.value();
  }
}

```

```

if (Serial.available()) {
  String cmd = Serial.readStringUntil('\n');
  cmd.trim();
  if (cmd.equalsIgnoreCase("RESET")) {
    Serial.println("· System Reset Command Received...");
    delay(500);
    softwareReset();
  } else if (cmd.equalsIgnoreCase("TEST")) {
    testMode = !testMode;
    Serial.print("· TEST MODE: ");
    Serial.println(testMode ? "ENABLED ✓" : "DISABLED ✗");
  }
}

if (millis() - lastCheck >= interval) {
  lastCheck = millis();
  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);

  float angle = abs(atan2(a.acceleration.y, a.acceleration.z) * 180 / PI);
  float vibration = analogRead(VIB_PIN);
  float temperature = temp.temperature;

  bool currentAccident = (angle > ANGLE_THRESHOLD) ||
    (vibration > VIB_THRESHOLD) ||
    (temperature > TEMP_THRESHOLD);

  if (currentAccident) {
    accidentDetected = true;
  }

  Serial.println("\n=====");
  Serial.print("Temperature : "); Serial.print(temperature); Serial.println(" °C");
  Serial.print("Tilt Angle : "); Serial.print(angle); Serial.println(" °");
  Serial.print("Vibration : "); Serial.println(vibration);
  Serial.print("Satellites : "); Serial.println(sats);

  if (gpsFixed) {
    Serial.print("Latitude : "); Serial.println(lat, 6);
    Serial.print("Longitude : "); Serial.println(lon, 6);
  } else {
    Serial.println("GPS Status : Searching...");
  }
}

```

```

Serial.print("STATUS    : ");
if (accidentDetected) {
    Serial.println(". ACCIDENT DETECTED .");
} else {
    Serial.println("NORMAL");
    digitalWrite(LED_PIN, LOW);
}
}

if (accidentDetected || testMode) {
    if (millis() - lastBlink >= blinkInterval) {
        lastBlink = millis();
        ledState = !ledState;
        digitalWrite(LED_PIN, ledState);
    }

    if (millis() - lastSend >= sendInterval) {
        lastSend = millis();
        sendLoRaAlert();
    }
}

// ----- LoRa Send Function -----
void sendLoRaAlert() {
    String packet = "ACCIDENT," + String(lat, 6) + "," + String(lon, 6);
    Serial.println("\n. Sending LoRa packet...");
    Serial.print(". Data: ");
    Serial.println(packet);

    LoRa.beginPacket();
    LoRa.print(packet);
    int status = LoRa.endPacket();

    if (status == 1)
        Serial.println("✓LoRa Packet Sent Successfully!");
    else
        Serial.println("✗LoRa Packet Failed to Send!");

    digitalWrite(LED_PIN, HIGH);
    delay(150);
    digitalWrite(LED_PIN, LOW);
}

```

```
// ----- Software Reset Function -----
void softwareReset() {
  Serial.println("· Restarting System...");
  delay(1000);
  asm volatile ("jmp 0");
}
```

10.2 Receiving Code:

```
#include <SPI.h>
#include <LoRa.h>
#include <SoftwareSerial.h>

// ----- LoRa Pins -----
#define LORA_SS 10
#define LORA_RST 9
#define LORA_DIO0 2

// ----- GSM (SIM900A) -----
SoftwareSerial gsm(4, 3); // RX, TX (D4 -> TX of GSM, D3 -> RX of GSM)

// ----- UART to ESP8266 -----
SoftwareSerial espSerial(8, 7); // RX, TX (D8 -> RX from ESP, D7 -> TX to ESP)

#define LED_PIN 6

void setup() {
  Serial.begin(9600);
  gsm.begin(9600);
  espSerial.begin(9600);
  delay(2000);

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  Serial.println("=====");
  Serial.println("· LoRa SX1278 Receiver + SIM900A + ESP8266");
  Serial.println("=====");

  // Initialize LoRa
  LoRa.setPins(LORA_SS, LORA_RST, LORA_DIO0);
  if (!LoRa.begin(433E6)) {
    Serial.println("✗LoRa init failed! Check wiring.");
    while (true);
  }
}
```

```

LoRa.setSyncWord(0xF3);
Serial.println("✓LoRa Receiver Ready!\n");

// Initialize GSM
sendAT("AT");
sendAT("AT+CMGF=1");
sendAT("AT+CSCS=\"GSM\"");
Serial.println("✓SIM900A Ready!\n");

// ESP connection
espSerial.println("✓ESP8266 Ready to receive data!");
}

void loop() {
  int packetSize = LoRa.parsePacket();
  if (packetSize) {
    String receivedData = "";
    while (LoRa.available()) {
      receivedData += (char)LoRa.read();
    }

    Serial.println("-----");
    Serial.println("• LoRa Packet Received!");
    Serial.print("• Message: ");
    Serial.println(receivedData);
    Serial.println("-----");

    // Extract GPS data
    int firstComma = receivedData.indexOf(',');
    int secondComma = receivedData.indexOf(',', firstComma + 1);
    String lat = receivedData.substring(firstComma + 1, secondComma);
    String lon = receivedData.substring(secondComma + 1);

    String message = "• Accident Detected!\nLocation: " + lat + "," + lon +
      "\nGoogle Maps: https://maps.google.com/?q=" + lat + "," + lon;

    sendSMS("+919003168321", message);
    sendSMS("+919940256556", message);
  }
  else if (receivedData.startsWith("DATA")) {
    //
    Serial.println("✓Normal Data Received.");
    digitalWrite(LED_PIN, LOW);
  }
}

```

Example:

DATA,Temp:31.5,Tilt:0.6,Vibration:150,Lat:10.7743,Lon:76.7211,Status:NORMAL

```

    else {
        Serial.println("· Unknown packet type");
    }
}
}

// ----- Helper Functions -----
void sendAT(String cmd) {
    gsm.println(cmd);
    delay(500);
    while (gsm.available()) {
        Serial.write(gsm.read());
    }
}

void sendSMS(String number, String text) {
    Serial.print("· Sending SMS to ");
    Serial.println(number);

    gsm.println("AT+CMGF=1");
    delay(500);
    gsm.print("AT+CMGS=\"");
    gsm.print(number);
    gsm.println("\"");
    delay(500);
    gsm.println(text);
    delay(500);
    gsm.write(26); // Ctrl+Z
    delay(5000);
    Serial.println("✓SMS Sent!\n");
}

```